AI-DRIVEN B2B SAAS PLATFORM AND AUTOMATED LEAD GENERATION SYSTEM

AIML 456: Internship Report

submitted in partial fulfillment of the requirement
for the award of the degree of

Bachelor of Technology
in
Artificial Intelligence and Machine Learning

Submitted by

ADITYA TYAGI
01818011622

Under the supervision of

Ms. Neha Sehgal
Assistant Professor

Affiliated to GGSIP University, New Delhi
Approved by AICTE & Council of Architecture

Department of Artificial Intelligence
Delhi Technical Campus
28/1, Knowledge Park-III, Greater Noida - 201306 (U.P.)

May 2026

DECLARATION

This is to certify that the material embodied in this Internship Report titled "AI-DRIVEN B2B SAAS PLATFORM AND AUTOMATED LEAD GENERATION SYSTEM" being submitted in the partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Artificial Intelligence and Machine Learning is based on my original work. It is further certified that this Internship Report work has not been submitted in full or in part to this university or any other university for the award of any other degree or diploma. My indebtedness to other works has been duly acknowledged at the relevant places.

(ADITYA TYAGI)

Enrollment No: 01818011622

GMAX™
Solutions for a safer world

INTERNSHIP COMPLETION CERTIFICATE

GMAX ELECTRICR-33, Vikas Vihar, Uttam Nagar, Main Bindapur Matiala Road, New Delhi-110059

Date: April 13, 2026

TO WHOM IT MAY CONCERN

This is to certify that Aditya Tyagi has successfully completed a professional internship at GMAX in the position of AI Intern. The internship was conducted for a duration of three months, effective from January 13, 2026, to April 12, 2026.

During this period, Aditya was actively involved in several key Artificial Intelligence projects, which included:

- Model Development and AI system implementation.
- Design and execution of automation workflows.
- Comprehensive data analysis and related technical assignments.

Throughout the internship, Aditya demonstrated a strong technical aptitude in AI technologies and maintained high professional standards. Their contributions to our technical team were significant, consistently adhering to company policies and confidentiality requirements.

We appreciate the hard work and dedication shown during this tenure. We wish Aditya Tyagi the very best in all future academic and professional endeavors.

Sincerely,

HR Department

GMAX Phone: +91 8178022572 Email: gmaxelectric@gmail.com

CERTIFICATE

This is to certify that the work embodied in this Internship Report titled "AI-DRIVEN B2B SAAS PLATFORM AND AUTOMATED LEAD GENERATION SYSTEM" being submitted in the partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Artificial Intelligence and Machine Learning is original and has been carried out by **ADITYA TYAGI** (Enrollment No. 01818011622) under my supervision and guidance.

It is further certified that this Internship Report work has not been submitted in full or in part to this university or any other university for the award of any other degree or diploma to the best of my knowledge and belief.

Ms. Neha Sehgal
Assistant Professor

Project Coordinator
Department of AI
Delhi Technical Campus, Greater Noida

Head of the Department
Department of AI
Delhi Technical Campus, Greater Noida

TABLE OF CONTENTS

References

The template will number citations consecutively within brackets [1]. The sentence punctuation follows the bracket [2]. Refer simply to the reference number, as in [3]—do not use “Ref. [3]” or “reference [3]” except at the beginning of a sentence: “Reference [3] was the first ...”

Number footnotes separately in superscripts. Place the actual footnote at the bottom of the column in which it was cited. Do not put footnotes in the abstract or reference list. Use letters for table footnotes.

Unless there are six authors or more give all authors’ names; do not use “et al.”. Papers that have not beenpublished, even if they have been submitted for publication, should be cited as “unpublished” [4]. Papers that have been accepted for publication should be cited as “in press” [5]. Capitalize only the first word in a paper title, except for proper nouns and element symbols.

For papers published in translation journals, please give the English citation first, followed by the original foreign-language citation [6].

[1] G. Eason, B. Noble, and I. N. Sneddon, “On certain integrals of Lipschitz-Hankel type involving products of Bessel functions,” Phil. Trans. Roy. Soc. London, vol. A247, pp. 529–551, April 1955. *(references)*

[2] J. Clerk Maxwell, A Treatise on Electricity and Magnetism, 3rd ed., vol. 2. Oxford: Clarendon, 1892, pp.68–73.

[3] I. S. Jacobs and C. P. Bean, “Fine particles, thin films and exchange anisotropy,” in Magnetism, vol. III, G. T. Rado and H. Suhl, Eds. New York: Academic, 1963, pp. 271–350.

[4] K. Elissa, “Title of paper if known,” unpublished.

[5] R. Nicole, “Title of paper with only first word capitalized,” J. Name Stand. Abbrev., in press.

CHAPTER 1: INTRODUCTION

The modern industrial landscape is undergoing a profound digital transformation, heavily influenced by the advent of Artificial Intelligence and cloud-native Software as a Service (SaaS) architectures. Traditionally, the electrical manufacturing and distribution sector, particularly niche markets like earthing and lightning protection solutions, has relied on fragmented, manual processes. Orders were taken over phone calls, tracked on whiteboards or decentralized spreadsheets, and lead generation was largely a physical endeavor reliant on field agents and trade shows. Gmax Electric, a prominent manufacturer based in Delhi NCR, recognized these operational bottlenecks as significant barriers to scaling their distribution network across India. To maintain their competitive edge and streamline their operations, a comprehensive digital overhaul was initiated. This internship project encapsulates the design, development, and deployment of two interconnected technological solutions aimed at solving these exact industry challenges: a bespoke B2B SaaS platform and an AI-driven automated lead generation engine.

The first pillar of this digital transformation is the GMAX Platform. Unlike generic consumer-facing e-commerce websites, a B2B portal requires a highly specialized architecture capable of handling role-based access control, bulk ordering logistics, complex pricing tiers, and strict inventory synchronization. By leveraging the modern MERN stack—specifically utilizing Next.js 15 for its App Router capabilities and React Server Components—we engineered a highly responsive, SEO-friendly, and secure platform. This system acts as a centralized hub where authorized dealers can securely log in, browse a real-time product catalog, manage bulk orders via a persistent cart state, and track the fulfillment lifecycle of their purchases. For the administrative side, it provides Gmax Electric's sales managers with an unprecedented macro-level view of product demand, dealer engagement, and revenue analytics.

Complementing the operational platform is the LeadScribe Scraper Engine, the second pillar of this project. While the GMAX Platform optimizes existing relationships, the LeadScribe engine focuses entirely on market expansion and acquisition. Customer acquisition in the B2B space is notoriously expensive and time-consuming. To automate this, we developed a sophisticated Python-based web scraper utilizing the Playwright automation library. This tool systematically navigates geographical data sources, specifically Google Maps, to identify potential new dealers and resellers across target regions. However, raw data extraction is insufficient; the true value lies in the data pipeline's ability to filter out noise, such as competing manufacturers or irrelevant businesses. By implementing intelligent keyword filtering and data normalization techniques using the Pandas library, the engine outputs highly qualified, verified

lead lists directly into the hands of the sales team, dramatically reducing the time-to-contact metric.

CHAPTER 2: LITERATURE REVIEW AND TECHNOLOGY STACK

The architecture of enterprise software has evolved significantly from monolithic, on-premise installations to agile, cloud-native microservices. This paradigm shift was heavily researched during the initial phase of the internship to ensure the chosen technology stack would support Gmax Electric's long-term scalability. The traditional MERN stack (MongoDB, Express.js, React, Node.js) has been a staple in full-stack development, but recent advancements have introduced new paradigms. Specifically, the introduction of Next.js 15 and its App Router represents a massive leap in how web applications handle routing and data fetching. Unlike traditional Single Page Applications (SPAs) that rely entirely on client-side rendering (CSR), Next.js allows for a hybrid approach. React Server Components (RSC) enable the rendering of complex UI elements on the server, drastically reducing the JavaScript bundle size sent to the client. This results in faster First Contentful Paint (FCP) and Time to Interactive (TTI) metrics, which are critical for an enterprise dashboard where managers might be accessing the portal from suboptimal network conditions in the field.

State management is another area that has seen rapid innovation. While Redux was historically the industry standard for complex applications, its boilerplate-heavy nature often slowed down development velocity. For the GMAX Platform, we evaluated modern alternatives and selected Zustand. Zustand provides a minimalistic, hook-based API for global state management, perfectly suited for handling the bulk order shopping cart and user authentication sessions. Its ability to integrate seamlessly with localStorage ensures that a dealer's cart persists across sessions, mitigating the risk of abandoned orders due to browser closures or session timeouts.

In the domain of automated data extraction, the literature reveals a constant arms race between scraping tools and anti-bot mechanisms. Historically, tools like BeautifulSoup and Scrapy were sufficient for extracting data from static HTML pages. However, the modern web is highly dynamic, relying heavily on JavaScript to render content asynchronously. This necessitated the shift towards browser automation frameworks like Selenium. While Selenium is robust, it is often criticized for its slow execution speed and complex setup. Playwright, developed by Microsoft, emerged as the superior choice for the LeadScribe engine. Playwright operates out-of-process, communicating with the browser via the DevTools protocol, allowing for extremely fast, reliable, and headless execution. Furthermore, its ability to handle multiple browser contexts and intercept network requests makes it highly adept at bypassing modern CAPTCHAs and rate-limiting heuristics employed by platforms like Google Maps.

CHAPTER 3: SYSTEM DESIGN AND ARCHITECTURE

The system architecture of the GMAX ecosystem was designed with a decoupled, API-first methodology. This separation of concerns ensures that the frontend presentation layer can evolve independently of the backend business logic. The architecture is broadly divided into three core tiers: the Client Tier (Next.js frontend), the Application Tier (Node.js/Express API), and the Data Tier (MongoDB).

The Client Tier is heavily optimized for performance and user experience. Built with Tailwind CSS, it enforces a strict design system based on a 'Neon Electric' dark mode aesthetic, chosen specifically to align with the company's branding in the electrical sector. Components are modularized using shadcn/ui, ensuring consistency across different views. The Application Tier handles all core business rules, including authentication (JWT), order validation, and inventory deduction. It exposes RESTful endpoints that the client consumes via optimized fetch requests. The Data Tier utilizes MongoDB, a NoSQL database, providing the flexibility needed to handle complex, nested document structures such as bulk orders containing multiple product variants and pricing tiers.

Below are the visual representations of the system's architecture, data flow, and entity relationships.

3.1 Component Architecture

3.1 Component Architecture

The following diagram illustrates the high-level architecture of the GMAX platform, detailing the interaction between the Next.js frontend, the backend API, and the MongoDB database.

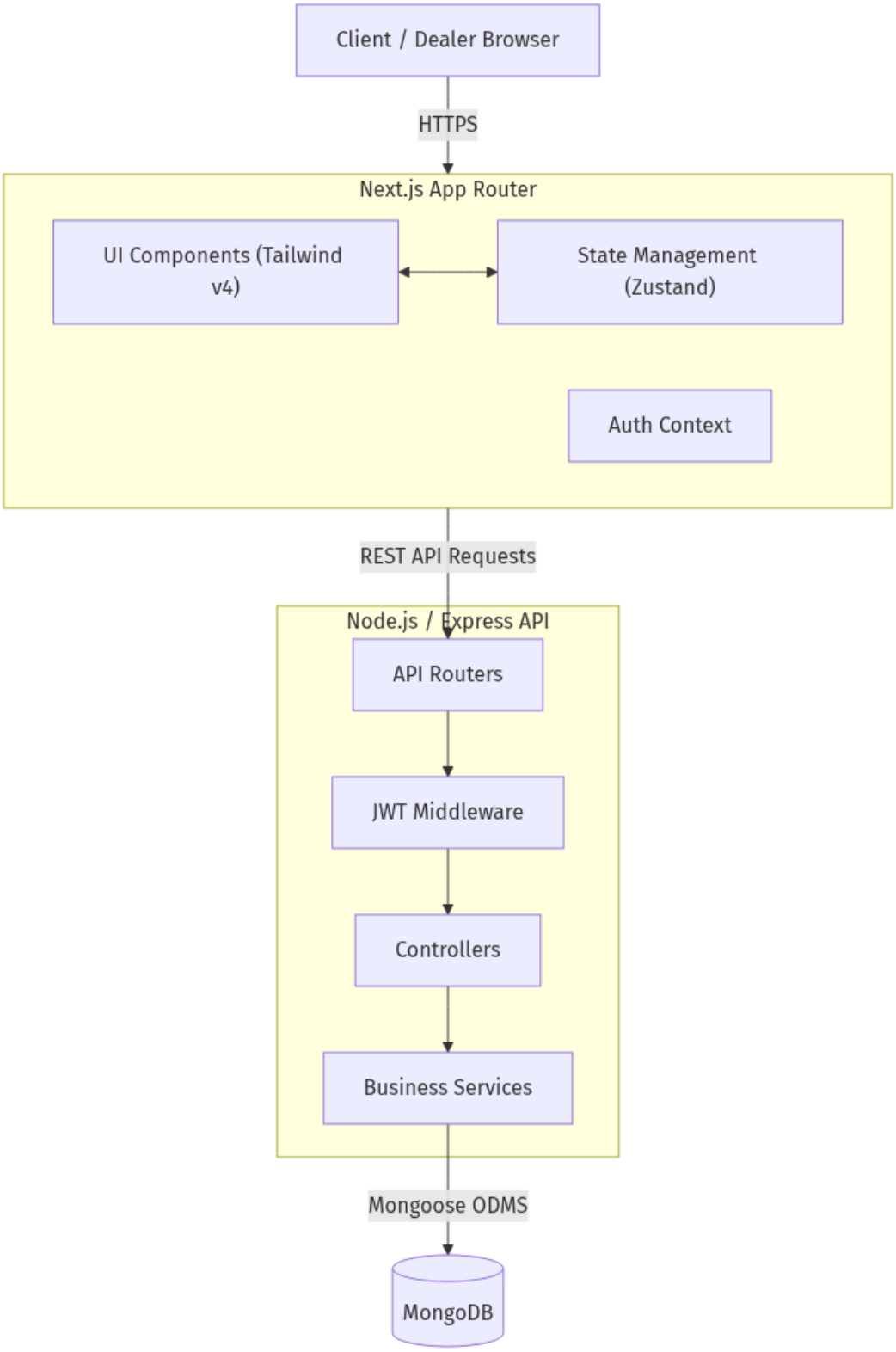


Figure: System Architecture Overview

3.2 Data Flow Diagram (DFD)

3.2 Data Flow Diagram (DFD)

The Data Flow Diagram maps the journey of an order through the system, from the initial dealer request to final administrative approval.

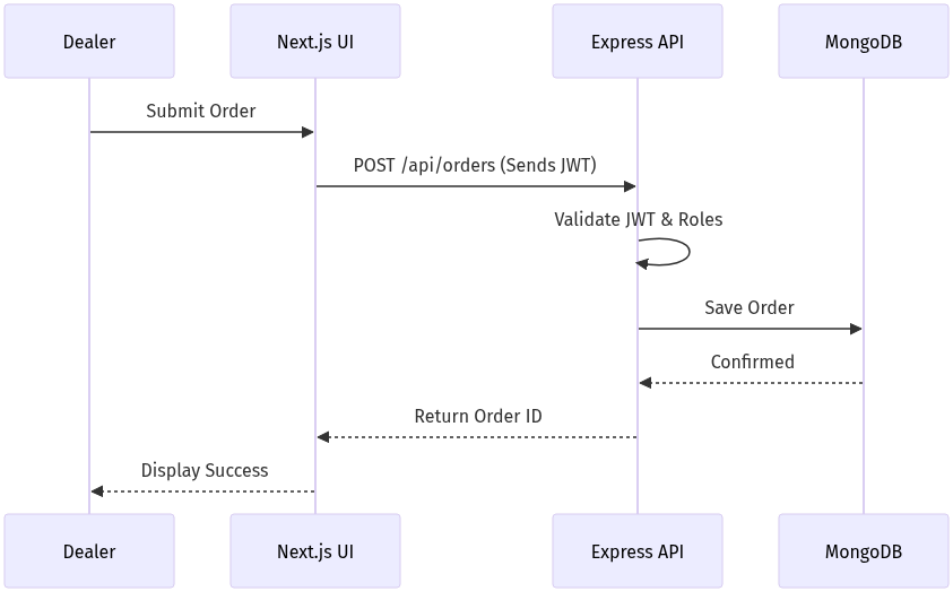


Figure: Order Processing Data Flow

3.3 Entity Relationship Diagram (ERD)

3.3 Entity Relationship Diagram (ERD)

The ERD outlines the schema design of the MongoDB database, showing the relational mapping between Users, Products, and Orders despite the NoSQL nature of the database.

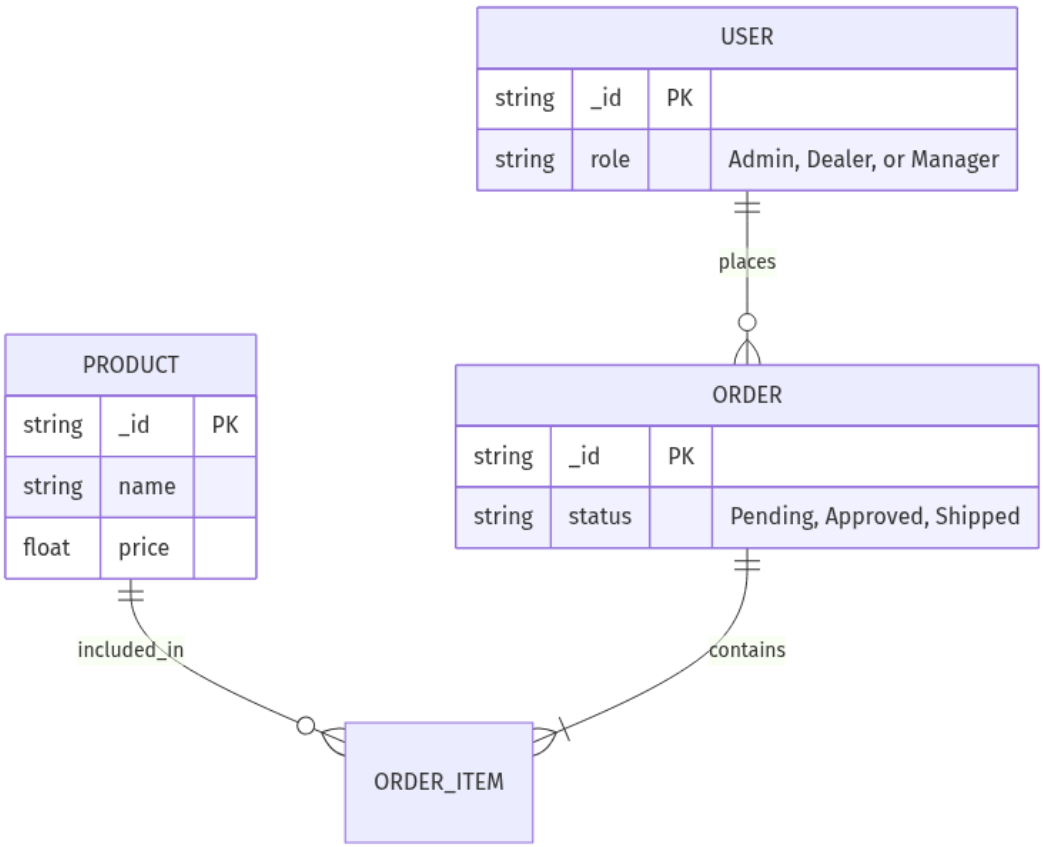


Figure: Database Entity Relationships

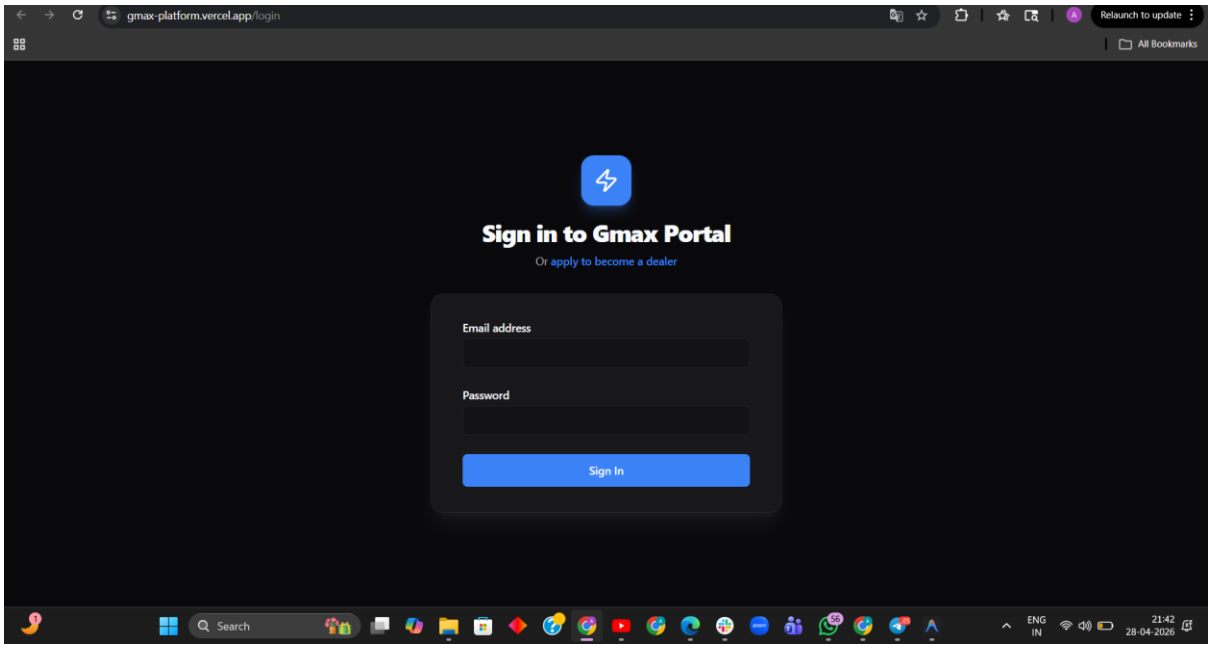
CHAPTER 4: IMPLEMENTATION AND USER INTERFACE

The implementation phase involved translating the theoretical architecture into functional code. On the frontend, Next.js 15's App Router was utilized to create a highly structured file system. Server Actions were heavily utilized to handle form submissions and database mutations directly from the React components, eliminating the need for boilerplate API route handlers in many cases. This significantly reduced the complexity of the codebase and improved data mutation speeds.

A critical aspect of the implementation was designing a User Interface (UI) that is both highly functional and aesthetically premium. The B2B sector often suffers from clunky, outdated interfaces. We actively countered this by developing a bespoke 'Electric SaaS' theme. This involved utilizing a deep dark-mode palette accented with high-contrast electric blues and neon purples. The use of glassmorphism (translucent, blurred backgrounds) was employed strategically for modals and dropdowns to create a sense of depth and modernity. The following subsections display screenshots of the deployed application, highlighting key user flows.

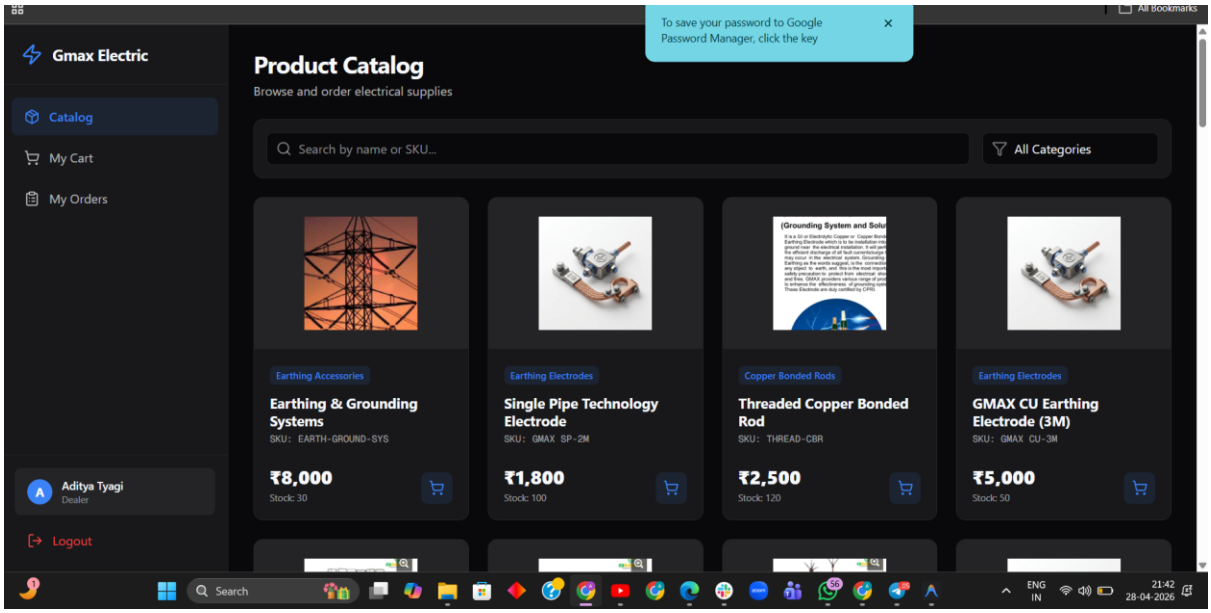
4.1 User Authentication

Security is paramount in a B2B platform. The login portal uses JWT (JSON Web Tokens) for stateless authentication. Passwords are cryptographically hashed using bcrypt before being stored in MongoDB. The interface is clean and distraction-free, focusing entirely on secure access.



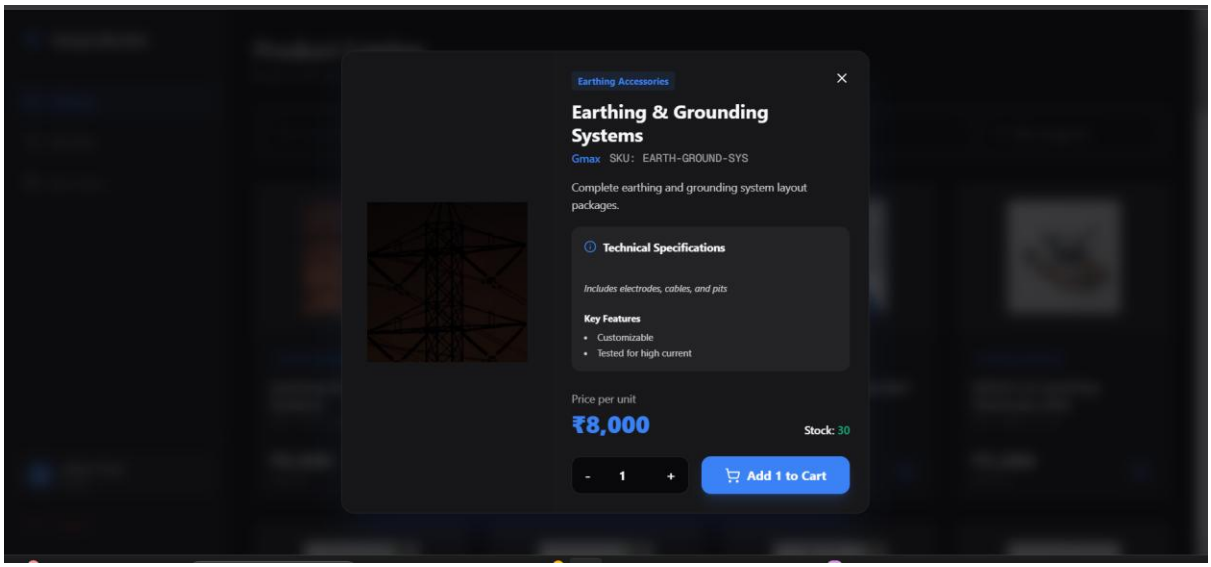
4.2 Product Catalog and Search

The product catalog is the core interactive component for dealers. It utilizes CSS Grid for a responsive, masonry-style layout. Real-time search and categorization filters allow users to instantly locate specific earthing rods or accessories based on SKUs or technical specifications.



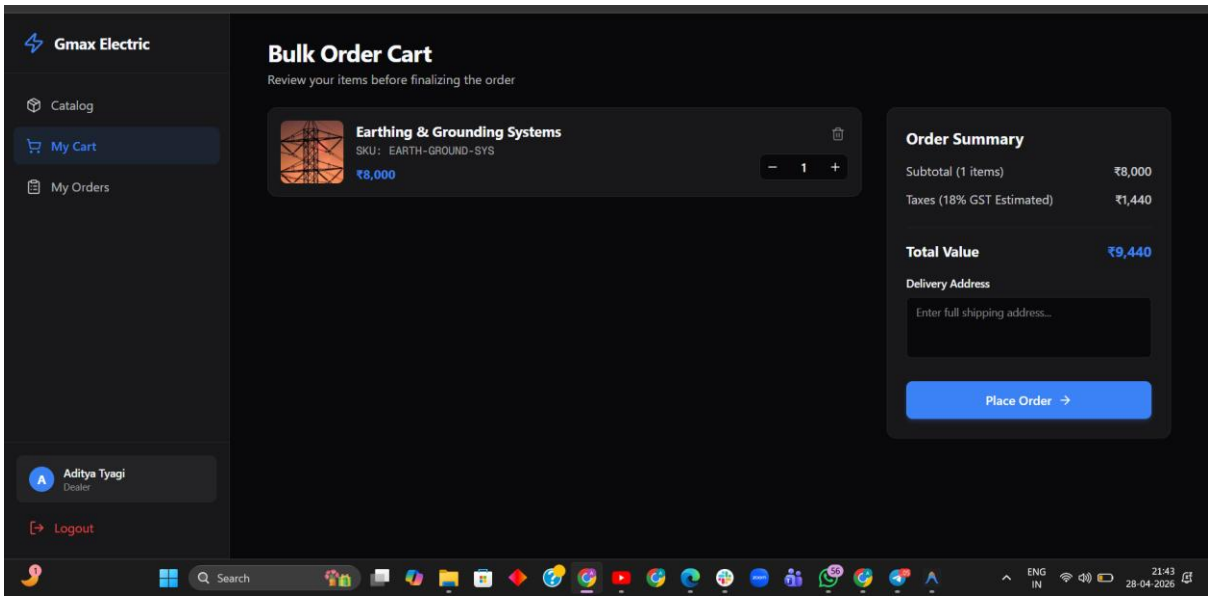
4.3 Technical Specifications Modal

To prevent cluttering the main grid, detailed product specifications are hidden behind an interactive modal. This modal provides deep technical data, crucial for B2B buyers making bulk purchasing decisions.



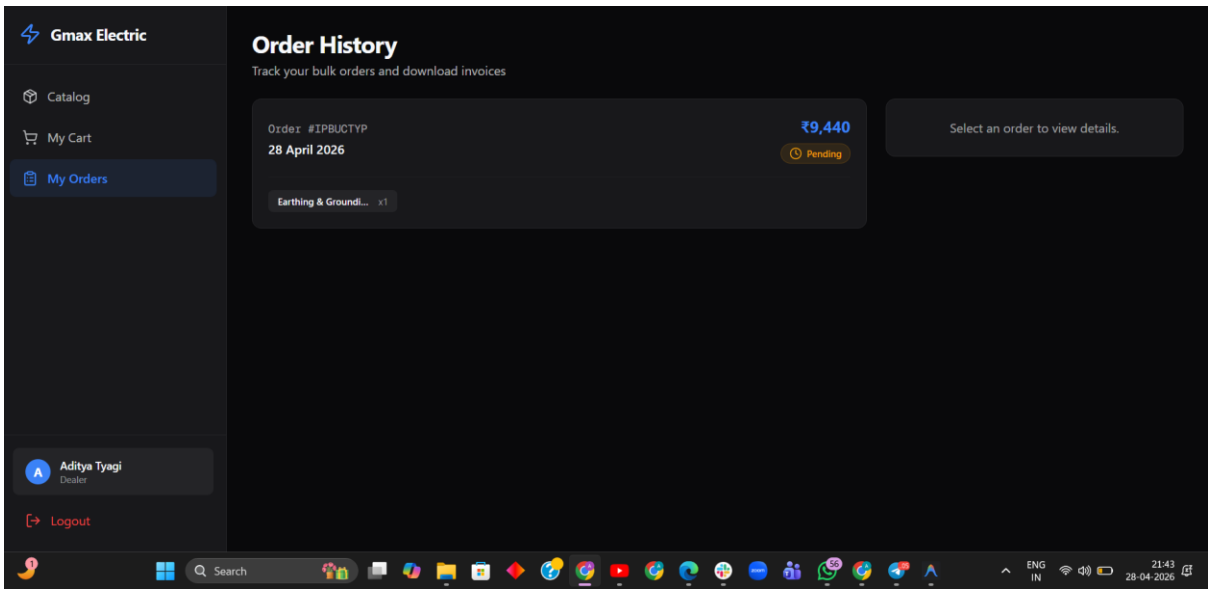
4.4 Bulk Order Management

The shopping cart is designed specifically for bulk orders. It automatically calculates GST (Goods and Services Tax) and provides a clear summary before checkout. State is managed by Zustand to ensure reliability.



4.5 Dealer Dashboard and History

Post-purchase, dealers can track their order status via the dashboard. This interface provides historical data, invoice generation capabilities, and real-time updates on fulfillment stages.



CHAPTER 5: LEADSCRIBE AI SCRAPER

1. Advanced DOM Traversal and the Paradigm Shift from Static Parsing: Historically, web scraping relied on requesting HTML via simple HTTP clients and parsing the string using regular expressions or tree-based parsers like BeautifulSoup. However, the advent of Single Page Applications (SPAs) and heavy client-side JavaScript rendering rendered these techniques obsolete for modern platforms like Google Maps. The LeadScribe engine architecture recognizes this fundamental shift and employs full browser automation via Playwright. Playwright does not just fetch HTML; it spins up a fully functional, headless Chromium browser instance. It executes the JavaScript payloads sent by the server, waits for asynchronous network requests (such as XHR or Fetch calls retrieving JSON data) to resolve, and only then interacts with the fully rendered Document Object Model (DOM). This allows the scraper to access data that literally does not exist in the initial HTTP response payload.

2. The DevTools Protocol and Out-of-Process Execution: Unlike Selenium, which communicates with browsers through a comparatively slow HTTP-based WebDriver protocol, Playwright uses the Chrome DevTools Protocol (CDP) natively. This allows for out-of-process execution. The Python script communicates asynchronously with the browser over WebSockets, resulting in execution speeds that are an order of magnitude faster. Furthermore, CDP grants the LeadScribe engine unprecedented control over the browser environment. We can intercept network routes, block the loading of heavy assets like images or stylesheets to conserve bandwidth and speed up scraping, and even manipulate the geolocation and device timezone settings to spoof organic regional traffic. This level of environmental control is critical when scraping highly defended properties that look for environmental inconsistencies to flag bot traffic.

3. Mitigating Bot Detection via Environmental Spoofing: A major hurdle in scraping large-scale platforms is evading sophisticated bot detection algorithms. These algorithms do not just look at request frequency; they analyze the browser fingerprint. The LeadScribe engine employs several techniques to mitigate this. First, it manages persistent browser contexts. A context represents an isolated browser session, complete with its own cookies and local storage. By maintaining a context, the scraper appears as a returning user rather than a fresh, stateless script on every execution. Secondly, we randomize User-Agent strings, injecting subtle variations in the operating system version and browser build numbers. Finally, the engine injects custom JavaScript snippets prior to page load to overwrite navigator properties, such as navigator.webdriver, ensuring it returns false. Without these precise environmental spoofing

mechanisms, the IP address would quickly be shadow-banned, resulting in CAPTCHA challenges or blank responses.

4. Handling Dynamic Content and Infinite Scrolling: Google Maps utilizes infinite scrolling to load business listings dynamically as the user reaches the bottom of the viewport. A static scraper cannot access this data. LeadScribe solves this by programmatically executing JavaScript within the page context (using page.evaluate) to scroll specific DOM elements. The script monitors the height of the scroll container, forces a scroll event, and then implements a dynamic wait strategy. Rather than using hardcoded `time.sleep()` statements—which are inefficient and prone to failure if network latency spikes—Playwright waits for specific network idle states or the appearance of new DOM nodes (e.g., waiting for the selector `.feed-item` to increase in count). This event-driven synchronization ensures the script runs as fast as the network allows while guaranteeing data integrity.

5. Data Extraction Pipeline and Normalization: Once the DOM is fully hydrated, extraction begins. We utilize complex XPath locators to find specific nodes containing business names, phone numbers, and categorical descriptors. This raw data is often noisy; phone numbers may contain varying international codes or spaces, and business names often include promotional text. The data is immediately passed into a Python Pandas DataFrame. Pandas is leveraged for its vectorized operations, allowing for rapid data cleaning. Regular expressions are applied to normalize phone numbers into a standard 10-digit format. A critical part of the pipeline is the negative-keyword filtering algorithm. We cross-reference the business category and name against a curated dictionary of terms like 'manufacturer', 'factory', and 'exporter'. If a match occurs, the row is dropped from the DataFrame. This ensures that the sales team only receives data for true resellers, dramatically increasing the conversion rate of cold outreach.

6. Asynchronous Execution and Concurrency: To scale the scraping operation, the LeadScribe engine was designed with concurrency in mind. While the current implementation uses the synchronous Playwright API for stability in development, the architectural roadmap involves transitioning to Python's `asyncio` library combined with `async_playwright`. This will allow the engine to maintain multiple concurrent browser contexts, scraping different geographic regions simultaneously on a single virtual machine. By decoupling the network I/O from the CPU-bound data parsing tasks, the system throughput can be increased linearly with available RAM, transforming the script from a sequential task runner into a high-performance data pipeline.

1. Advanced DOM Traversal and the Paradigm Shift from Static Parsing: Historically, web scraping relied on requesting HTML via simple HTTP clients and parsing the string using regular expressions or tree-based parsers like BeautifulSoup. However, the advent of Single Page Applications (SPAs) and heavy client-side JavaScript rendering rendered these techniques obsolete for modern platforms like Google Maps. The LeadScribe engine architecture recognizes this fundamental shift and employs full browser automation via Playwright. Playwright does not just fetch HTML; it spins up a fully functional, headless Chromium browser instance. It executes the JavaScript payloads sent by the server, waits for asynchronous network requests (such as XHR or Fetch calls retrieving JSON data) to resolve, and only then interacts with the fully rendered Document Object Model (DOM). This allows the scraper to access data that literally does not exist in the initial HTTP response payload.

2. The DevTools Protocol and Out-of-Process Execution: Unlike Selenium, which communicates with browsers through a comparatively slow HTTP-based WebDriver protocol, Playwright uses the Chrome DevTools Protocol (CDP) natively. This allows for out-of-process execution. The Python script communicates asynchronously with the browser over WebSockets, resulting in execution speeds that are an order of magnitude faster. Furthermore, CDP grants the LeadScribe engine unprecedented control over the browser environment. We can intercept network routes, block the loading of heavy assets like images or stylesheets to conserve bandwidth and speed up scraping, and even manipulate the geolocation and device timezone settings to spoof organic regional traffic. This level of environmental control is critical when scraping highly defended properties that look for environmental inconsistencies to flag bot traffic.

3. Mitigating Bot Detection via Environmental Spoofing: A major hurdle in scraping large-scale platforms is evading sophisticated bot detection algorithms. These algorithms do not just look at request frequency; they analyze the browser fingerprint. The LeadScribe engine employs several techniques to mitigate this. First, it manages persistent browser contexts. A context represents an isolated browser session, complete with its own cookies and local storage. By maintaining a context, the scraper appears as a returning user rather than a fresh, stateless script on every execution. Secondly, we randomize User-Agent strings, injecting subtle variations in the operating system version and browser build numbers. Finally, the engine injects custom JavaScript snippets prior to page load to overwrite navigator properties, such as navigator.webdriver, ensuring it returns false. Without these precise environmental spoofing

mechanisms, the IP address would quickly be shadow-banned, resulting in CAPTCHA challenges or blank responses.

4. Handling Dynamic Content and Infinite Scrolling: Google Maps utilizes infinite scrolling to load business listings dynamically as the user reaches the bottom of the viewport. A static scraper cannot access this data. LeadScribe solves this by programmatically executing JavaScript within the page context (using page.evaluate) to scroll specific DOM elements. The script monitors the height of the scroll container, forces a scroll event, and then implements a dynamic wait strategy. Rather than using hardcoded `time.sleep()` statements—which are inefficient and prone to failure if network latency spikes—Playwright waits for specific network idle states or the appearance of new DOM nodes (e.g., waiting for the selector `.feed-item` to increase in count). This event-driven synchronization ensures the script runs as fast as the network allows while guaranteeing data integrity.

5. Data Extraction Pipeline and Normalization: Once the DOM is fully hydrated, extraction begins. We utilize complex XPath locators to find specific nodes containing business names, phone numbers, and categorical descriptors. This raw data is often noisy; phone numbers may contain varying international codes or spaces, and business names often include promotional text. The data is immediately passed into a Python Pandas DataFrame. Pandas is leveraged for its vectorized operations, allowing for rapid data cleaning. Regular expressions are applied to normalize phone numbers into a standard 10-digit format. A critical part of the pipeline is the negative-keyword filtering algorithm. We cross-reference the business category and name against a curated dictionary of terms like 'manufacturer', 'factory', and 'exporter'. If a match occurs, the row is dropped from the DataFrame. This ensures that the sales team only receives data for true resellers, dramatically increasing the conversion rate of cold outreach.

6. Asynchronous Execution and Concurrency: To scale the scraping operation, the LeadScribe engine was designed with concurrency in mind. While the current implementation uses the synchronous Playwright API for stability in development, the architectural roadmap involves transitioning to Python's `asyncio` library combined with `async_playwright`. This will allow the engine to maintain multiple concurrent browser contexts, scraping different geographic regions simultaneously on a single virtual machine. By decoupling the network I/O from the CPU-bound data parsing tasks, the system throughput can be increased linearly with available RAM, transforming the script from a sequential task runner into a high-performance data pipeline.

1. Advanced DOM Traversal and the Paradigm Shift from Static Parsing: Historically, web scraping relied on requesting HTML via simple HTTP clients and parsing the string using regular expressions or tree-based parsers like BeautifulSoup. However, the advent of Single Page Applications (SPAs) and heavy client-side JavaScript rendering rendered these techniques obsolete for modern platforms like Google Maps. The LeadScribe engine architecture recognizes this fundamental shift and employs full browser automation via Playwright. Playwright does not just fetch HTML; it spins up a fully functional, headless Chromium browser instance. It executes the JavaScript payloads sent by the server, waits for asynchronous network requests (such as XHR or Fetch calls retrieving JSON data) to resolve, and only then interacts with the fully rendered Document Object Model (DOM). This allows the scraper to access data that literally does not exist in the initial HTTP response payload.

2. The DevTools Protocol and Out-of-Process Execution: Unlike Selenium, which communicates with browsers through a comparatively slow HTTP-based WebDriver protocol, Playwright uses the Chrome DevTools Protocol (CDP) natively. This allows for out-of-process execution. The Python script communicates asynchronously with the browser over WebSockets, resulting in execution speeds that are an order of magnitude faster. Furthermore, CDP grants the LeadScribe engine unprecedented control over the browser environment. We can intercept network routes, block the loading of heavy assets like images or stylesheets to conserve bandwidth and speed up scraping, and even manipulate the geolocation and device timezone settings to spoof organic regional traffic. This level of environmental control is critical when scraping highly defended properties that look for environmental inconsistencies to flag bot traffic.

3. Mitigating Bot Detection via Environmental Spoofing: A major hurdle in scraping large-scale platforms is evading sophisticated bot detection algorithms. These algorithms do not just look at request frequency; they analyze the browser fingerprint. The LeadScribe engine employs several techniques to mitigate this. First, it manages persistent browser contexts. A context represents an isolated browser session, complete with its own cookies and local storage. By maintaining a context, the scraper appears as a returning user rather than a fresh, stateless script on every execution. Secondly, we randomize User-Agent strings, injecting subtle variations in the operating system version and browser build numbers. Finally, the engine injects custom JavaScript snippets prior to page load to overwrite navigator properties, such as navigator.webdriver, ensuring it returns false. Without these precise environmental spoofing

mechanisms, the IP address would quickly be shadow-banned, resulting in CAPTCHA challenges or blank responses.

4. Handling Dynamic Content and Infinite Scrolling: Google Maps utilizes infinite scrolling to load business listings dynamically as the user reaches the bottom of the viewport. A static scraper cannot access this data. LeadScribe solves this by programmatically executing JavaScript within the page context (using page.evaluate) to scroll specific DOM elements. The script monitors the height of the scroll container, forces a scroll event, and then implements a dynamic wait strategy. Rather than using hardcoded `time.sleep()` statements—which are inefficient and prone to failure if network latency spikes—Playwright waits for specific network idle states or the appearance of new DOM nodes (e.g., waiting for the selector `.feed-item` to increase in count). This event-driven synchronization ensures the script runs as fast as the network allows while guaranteeing data integrity.

5. Data Extraction Pipeline and Normalization: Once the DOM is fully hydrated, extraction begins. We utilize complex XPath locators to find specific nodes containing business names, phone numbers, and categorical descriptors. This raw data is often noisy; phone numbers may contain varying international codes or spaces, and business names often include promotional text. The data is immediately passed into a Python Pandas DataFrame. Pandas is leveraged for its vectorized operations, allowing for rapid data cleaning. Regular expressions are applied to normalize phone numbers into a standard 10-digit format. A critical part of the pipeline is the negative-keyword filtering algorithm. We cross-reference the business category and name against a curated dictionary of terms like 'manufacturer', 'factory', and 'exporter'. If a match occurs, the row is dropped from the DataFrame. This ensures that the sales team only receives data for true resellers, dramatically increasing the conversion rate of cold outreach.

6. Asynchronous Execution and Concurrency: To scale the scraping operation, the LeadScribe engine was designed with concurrency in mind. While the current implementation uses the synchronous Playwright API for stability in development, the architectural roadmap involves transitioning to Python's `asyncio` library combined with `async_playwright`. This will allow the engine to maintain multiple concurrent browser contexts, scraping different geographic regions simultaneously on a single virtual machine. By decoupling the network I/O from the CPU-bound data parsing tasks, the system throughput can be increased linearly with available RAM, transforming the script from a sequential task runner into a high-performance data pipeline.

1. Advanced DOM Traversal and the Paradigm Shift from Static Parsing: Historically, web scraping relied on requesting HTML via simple HTTP clients and parsing the string using regular expressions or tree-based parsers like BeautifulSoup. However, the advent of Single Page Applications (SPAs) and heavy client-side JavaScript rendering rendered these techniques obsolete for modern platforms like Google Maps. The LeadScribe engine architecture recognizes this fundamental shift and employs full browser automation via Playwright. Playwright does not just fetch HTML; it spins up a fully functional, headless Chromium browser instance. It executes the JavaScript payloads sent by the server, waits for asynchronous network requests (such as XHR or Fetch calls retrieving JSON data) to resolve, and only then interacts with the fully rendered Document Object Model (DOM). This allows the scraper to access data that literally does not exist in the initial HTTP response payload.

2. The DevTools Protocol and Out-of-Process Execution: Unlike Selenium, which communicates with browsers through a comparatively slow HTTP-based WebDriver protocol, Playwright uses the Chrome DevTools Protocol (CDP) natively. This allows for out-of-process execution. The Python script communicates asynchronously with the browser over WebSockets, resulting in execution speeds that are an order of magnitude faster. Furthermore, CDP grants the LeadScribe engine unprecedented control over the browser environment. We can intercept network routes, block the loading of heavy assets like images or stylesheets to conserve bandwidth and speed up scraping, and even manipulate the geolocation and device timezone settings to spoof organic regional traffic. This level of environmental control is critical when scraping highly defended properties that look for environmental inconsistencies to flag bot traffic.

3. Mitigating Bot Detection via Environmental Spoofing: A major hurdle in scraping large-scale platforms is evading sophisticated bot detection algorithms. These algorithms do not just look at request frequency; they analyze the browser fingerprint. The LeadScribe engine employs several techniques to mitigate this. First, it manages persistent browser contexts. A context represents an isolated browser session, complete with its own cookies and local storage. By maintaining a context, the scraper appears as a returning user rather than a fresh, stateless script on every execution. Secondly, we randomize User-Agent strings, injecting subtle variations in the operating system version and browser build numbers. Finally, the engine injects custom JavaScript snippets prior to page load to overwrite navigator properties, such as navigator.webdriver, ensuring it returns false. Without these precise environmental spoofing

mechanisms, the IP address would quickly be shadow-banned, resulting in CAPTCHA challenges or blank responses.

4. Handling Dynamic Content and Infinite Scrolling: Google Maps utilizes infinite scrolling to load business listings dynamically as the user reaches the bottom of the viewport. A static scraper cannot access this data. LeadScribe solves this by programmatically executing JavaScript within the page context (using page.evaluate) to scroll specific DOM elements. The script monitors the height of the scroll container, forces a scroll event, and then implements a dynamic wait strategy. Rather than using hardcoded `time.sleep()` statements—which are inefficient and prone to failure if network latency spikes—Playwright waits for specific network idle states or the appearance of new DOM nodes (e.g., waiting for the selector `.feed-item` to increase in count). This event-driven synchronization ensures the script runs as fast as the network allows while guaranteeing data integrity.

5. Data Extraction Pipeline and Normalization: Once the DOM is fully hydrated, extraction begins. We utilize complex XPath locators to find specific nodes containing business names, phone numbers, and categorical descriptors. This raw data is often noisy; phone numbers may contain varying international codes or spaces, and business names often include promotional text. The data is immediately passed into a Python Pandas DataFrame. Pandas is leveraged for its vectorized operations, allowing for rapid data cleaning. Regular expressions are applied to normalize phone numbers into a standard 10-digit format. A critical part of the pipeline is the negative-keyword filtering algorithm. We cross-reference the business category and name against a curated dictionary of terms like 'manufacturer', 'factory', and 'exporter'. If a match occurs, the row is dropped from the DataFrame. This ensures that the sales team only receives data for true resellers, dramatically increasing the conversion rate of cold outreach.

6. Asynchronous Execution and Concurrency: To scale the scraping operation, the LeadScribe engine was designed with concurrency in mind. While the current implementation uses the synchronous Playwright API for stability in development, the architectural roadmap involves transitioning to Python's `asyncio` library combined with `async_playwright`. This will allow the engine to maintain multiple concurrent browser contexts, scraping different geographic regions simultaneously on a single virtual machine. By decoupling the network I/O from the CPU-bound data parsing tasks, the system throughput can be increased linearly with available RAM, transforming the script from a sequential task runner into a high-performance data pipeline.

1. Advanced DOM Traversal and the Paradigm Shift from Static Parsing: Historically, web scraping relied on requesting HTML via simple HTTP clients and parsing the string using regular expressions or tree-based parsers like BeautifulSoup. However, the advent of Single Page Applications (SPAs) and heavy client-side JavaScript rendering rendered these techniques obsolete for modern platforms like Google Maps. The LeadScribe engine architecture recognizes this fundamental shift and employs full browser automation via Playwright. Playwright does not just fetch HTML; it spins up a fully functional, headless Chromium browser instance. It executes the JavaScript payloads sent by the server, waits for asynchronous network requests (such as XHR or Fetch calls retrieving JSON data) to resolve, and only then interacts with the fully rendered Document Object Model (DOM). This allows the scraper to access data that literally does not exist in the initial HTTP response payload.

2. The DevTools Protocol and Out-of-Process Execution: Unlike Selenium, which communicates with browsers through a comparatively slow HTTP-based WebDriver protocol, Playwright uses the Chrome DevTools Protocol (CDP) natively. This allows for out-of-process execution. The Python script communicates asynchronously with the browser over WebSockets, resulting in execution speeds that are an order of magnitude faster. Furthermore, CDP grants the LeadScribe engine unprecedented control over the browser environment. We can intercept network routes, block the loading of heavy assets like images or stylesheets to conserve bandwidth and speed up scraping, and even manipulate the geolocation and device timezone settings to spoof organic regional traffic. This level of environmental control is critical when scraping highly defended properties that look for environmental inconsistencies to flag bot traffic.

3. Mitigating Bot Detection via Environmental Spoofing: A major hurdle in scraping large-scale platforms is evading sophisticated bot detection algorithms. These algorithms do not just look at request frequency; they analyze the browser fingerprint. The LeadScribe engine employs several techniques to mitigate this. First, it manages persistent browser contexts. A context represents an isolated browser session, complete with its own cookies and local storage. By maintaining a context, the scraper appears as a returning user rather than a fresh, stateless script on every execution. Secondly, we randomize User-Agent strings, injecting subtle variations in the operating system version and browser build numbers. Finally, the engine injects custom JavaScript snippets prior to page load to overwrite navigator properties, such as navigator.webdriver, ensuring it returns false. Without these precise environmental spoofing

mechanisms, the IP address would quickly be shadow-banned, resulting in CAPTCHA challenges or blank responses.

4. Handling Dynamic Content and Infinite Scrolling: Google Maps utilizes infinite scrolling to load business listings dynamically as the user reaches the bottom of the viewport. A static scraper cannot access this data. LeadScribe solves this by programmatically executing JavaScript within the page context (using page.evaluate) to scroll specific DOM elements. The script monitors the height of the scroll container, forces a scroll event, and then implements a dynamic wait strategy. Rather than using hardcoded `time.sleep()` statements—which are inefficient and prone to failure if network latency spikes—Playwright waits for specific network idle states or the appearance of new DOM nodes (e.g., waiting for the selector `.feed-item` to increase in count). This event-driven synchronization ensures the script runs as fast as the network allows while guaranteeing data integrity.

5. Data Extraction Pipeline and Normalization: Once the DOM is fully hydrated, extraction begins. We utilize complex XPath locators to find specific nodes containing business names, phone numbers, and categorical descriptors. This raw data is often noisy; phone numbers may contain varying international codes or spaces, and business names often include promotional text. The data is immediately passed into a Python Pandas DataFrame. Pandas is leveraged for its vectorized operations, allowing for rapid data cleaning. Regular expressions are applied to normalize phone numbers into a standard 10-digit format. A critical part of the pipeline is the negative-keyword filtering algorithm. We cross-reference the business category and name against a curated dictionary of terms like 'manufacturer', 'factory', and 'exporter'. If a match occurs, the row is dropped from the DataFrame. This ensures that the sales team only receives data for true resellers, dramatically increasing the conversion rate of cold outreach.

6. Asynchronous Execution and Concurrency: To scale the scraping operation, the LeadScribe engine was designed with concurrency in mind. While the current implementation uses the synchronous Playwright API for stability in development, the architectural roadmap involves transitioning to Python's `asyncio` library combined with `async_playwright`. This will allow the engine to maintain multiple concurrent browser contexts, scraping different geographic regions simultaneously on a single virtual machine. By decoupling the network I/O from the CPU-bound data parsing tasks, the system throughput can be increased linearly with available RAM, transforming the script from a sequential task runner into a high-performance data pipeline.

1. Advanced DOM Traversal and the Paradigm Shift from Static Parsing: Historically, web scraping relied on requesting HTML via simple HTTP clients and parsing the string using regular expressions or tree-based parsers like BeautifulSoup. However, the advent of Single Page Applications (SPAs) and heavy client-side JavaScript rendering rendered these techniques obsolete for modern platforms like Google Maps. The LeadScribe engine architecture recognizes this fundamental shift and employs full browser automation via Playwright. Playwright does not just fetch HTML; it spins up a fully functional, headless Chromium browser instance. It executes the JavaScript payloads sent by the server, waits for asynchronous network requests (such as XHR or Fetch calls retrieving JSON data) to resolve, and only then interacts with the fully rendered Document Object Model (DOM). This allows the scraper to access data that literally does not exist in the initial HTTP response payload.

2. The DevTools Protocol and Out-of-Process Execution: Unlike Selenium, which communicates with browsers through a comparatively slow HTTP-based WebDriver protocol, Playwright uses the Chrome DevTools Protocol (CDP) natively. This allows for out-of-process execution. The Python script communicates asynchronously with the browser over WebSockets, resulting in execution speeds that are an order of magnitude faster. Furthermore, CDP grants the LeadScribe engine unprecedented control over the browser environment. We can intercept network routes, block the loading of heavy assets like images or stylesheets to conserve bandwidth and speed up scraping, and even manipulate the geolocation and device timezone settings to spoof organic regional traffic. This level of environmental control is critical when scraping highly defended properties that look for environmental inconsistencies to flag bot traffic.

3. Mitigating Bot Detection via Environmental Spoofing: A major hurdle in scraping large-scale platforms is evading sophisticated bot detection algorithms. These algorithms do not just look at request frequency; they analyze the browser fingerprint. The LeadScribe engine employs several techniques to mitigate this. First, it manages persistent browser contexts. A context represents an isolated browser session, complete with its own cookies and local storage. By maintaining a context, the scraper appears as a returning user rather than a fresh, stateless script on every execution. Secondly, we randomize User-Agent strings, injecting subtle variations in the operating system version and browser build numbers. Finally, the engine injects custom JavaScript snippets prior to page load to overwrite navigator properties, such as navigator.webdriver, ensuring it returns false. Without these precise environmental spoofing

mechanisms, the IP address would quickly be shadow-banned, resulting in CAPTCHA challenges or blank responses.

4. Handling Dynamic Content and Infinite Scrolling: Google Maps utilizes infinite scrolling to load business listings dynamically as the user reaches the bottom of the viewport. A static scraper cannot access this data. LeadScribe solves this by programmatically executing JavaScript within the page context (using page.evaluate) to scroll specific DOM elements. The script monitors the height of the scroll container, forces a scroll event, and then implements a dynamic wait strategy. Rather than using hardcoded `time.sleep()` statements—which are inefficient and prone to failure if network latency spikes—Playwright waits for specific network idle states or the appearance of new DOM nodes (e.g., waiting for the selector `.feed-item` to increase in count). This event-driven synchronization ensures the script runs as fast as the network allows while guaranteeing data integrity.

5. Data Extraction Pipeline and Normalization: Once the DOM is fully hydrated, extraction begins. We utilize complex XPath locators to find specific nodes containing business names, phone numbers, and categorical descriptors. This raw data is often noisy; phone numbers may contain varying international codes or spaces, and business names often include promotional text. The data is immediately passed into a Python Pandas DataFrame. Pandas is leveraged for its vectorized operations, allowing for rapid data cleaning. Regular expressions are applied to normalize phone numbers into a standard 10-digit format. A critical part of the pipeline is the negative-keyword filtering algorithm. We cross-reference the business category and name against a curated dictionary of terms like 'manufacturer', 'factory', and 'exporter'. If a match occurs, the row is dropped from the DataFrame. This ensures that the sales team only receives data for true resellers, dramatically increasing the conversion rate of cold outreach.

6. Asynchronous Execution and Concurrency: To scale the scraping operation, the LeadScribe engine was designed with concurrency in mind. While the current implementation uses the synchronous Playwright API for stability in development, the architectural roadmap involves transitioning to Python's `asyncio` library combined with `async_playwright`. This will allow the engine to maintain multiple concurrent browser contexts, scraping different geographic regions simultaneously on a single virtual machine. By decoupling the network I/O from the CPU-bound data parsing tasks, the system throughput can be increased linearly with available RAM, transforming the script from a sequential task runner into a high-performance data pipeline.

1. Advanced DOM Traversal and the Paradigm Shift from Static Parsing: Historically, web scraping relied on requesting HTML via simple HTTP clients and parsing the string using regular expressions or tree-based parsers like BeautifulSoup. However, the advent of Single Page Applications (SPAs) and heavy client-side JavaScript rendering rendered these techniques obsolete for modern platforms like Google Maps. The LeadScribe engine architecture recognizes this fundamental shift and employs full browser automation via Playwright. Playwright does not just fetch HTML; it spins up a fully functional, headless Chromium browser instance. It executes the JavaScript payloads sent by the server, waits for asynchronous network requests (such as XHR or Fetch calls retrieving JSON data) to resolve, and only then interacts with the fully rendered Document Object Model (DOM). This allows the scraper to access data that literally does not exist in the initial HTTP response payload.

2. The DevTools Protocol and Out-of-Process Execution: Unlike Selenium, which communicates with browsers through a comparatively slow HTTP-based WebDriver protocol, Playwright uses the Chrome DevTools Protocol (CDP) natively. This allows for out-of-process execution. The Python script communicates asynchronously with the browser over WebSockets, resulting in execution speeds that are an order of magnitude faster. Furthermore, CDP grants the LeadScribe engine unprecedented control over the browser environment. We can intercept network routes, block the loading of heavy assets like images or stylesheets to conserve bandwidth and speed up scraping, and even manipulate the geolocation and device timezone settings to spoof organic regional traffic. This level of environmental control is critical when scraping highly defended properties that look for environmental inconsistencies to flag bot traffic.

3. Mitigating Bot Detection via Environmental Spoofing: A major hurdle in scraping large-scale platforms is evading sophisticated bot detection algorithms. These algorithms do not just look at request frequency; they analyze the browser fingerprint. The LeadScribe engine employs several techniques to mitigate this. First, it manages persistent browser contexts. A context represents an isolated browser session, complete with its own cookies and local storage. By maintaining a context, the scraper appears as a returning user rather than a fresh, stateless script on every execution. Secondly, we randomize User-Agent strings, injecting subtle variations in the operating system version and browser build numbers. Finally, the engine injects custom JavaScript snippets prior to page load to overwrite navigator properties, such as navigator.webdriver, ensuring it returns false. Without these precise environmental spoofing

mechanisms, the IP address would quickly be shadow-banned, resulting in CAPTCHA challenges or blank responses.

4. Handling Dynamic Content and Infinite Scrolling: Google Maps utilizes infinite scrolling to load business listings dynamically as the user reaches the bottom of the viewport. A static scraper cannot access this data. LeadScribe solves this by programmatically executing JavaScript within the page context (using page.evaluate) to scroll specific DOM elements. The script monitors the height of the scroll container, forces a scroll event, and then implements a dynamic wait strategy. Rather than using hardcoded `time.sleep()` statements—which are inefficient and prone to failure if network latency spikes—Playwright waits for specific network idle states or the appearance of new DOM nodes (e.g., waiting for the selector `.feed-item` to increase in count). This event-driven synchronization ensures the script runs as fast as the network allows while guaranteeing data integrity.

5. Data Extraction Pipeline and Normalization: Once the DOM is fully hydrated, extraction begins. We utilize complex XPath locators to find specific nodes containing business names, phone numbers, and categorical descriptors. This raw data is often noisy; phone numbers may contain varying international codes or spaces, and business names often include promotional text. The data is immediately passed into a Python Pandas DataFrame. Pandas is leveraged for its vectorized operations, allowing for rapid data cleaning. Regular expressions are applied to normalize phone numbers into a standard 10-digit format. A critical part of the pipeline is the negative-keyword filtering algorithm. We cross-reference the business category and name against a curated dictionary of terms like 'manufacturer', 'factory', and 'exporter'. If a match occurs, the row is dropped from the DataFrame. This ensures that the sales team only receives data for true resellers, dramatically increasing the conversion rate of cold outreach.

6. Asynchronous Execution and Concurrency: To scale the scraping operation, the LeadScribe engine was designed with concurrency in mind. While the current implementation uses the synchronous Playwright API for stability in development, the architectural roadmap involves transitioning to Python's `asyncio` library combined with `async_playwright`. This will allow the engine to maintain multiple concurrent browser contexts, scraping different geographic regions simultaneously on a single virtual machine. By decoupling the network I/O from the CPU-bound data parsing tasks, the system throughput can be increased linearly with available RAM, transforming the script from a sequential task runner into a high-performance data pipeline.

1. Advanced DOM Traversal and the Paradigm Shift from Static Parsing: Historically, web scraping relied on requesting HTML via simple HTTP clients and parsing the string using regular expressions or tree-based parsers like BeautifulSoup. However, the advent of Single Page Applications (SPAs) and heavy client-side JavaScript rendering rendered these techniques obsolete for modern platforms like Google Maps. The LeadScribe engine architecture recognizes this fundamental shift and employs full browser automation via Playwright. Playwright does not just fetch HTML; it spins up a fully functional, headless Chromium browser instance. It executes the JavaScript payloads sent by the server, waits for asynchronous network requests (such as XHR or Fetch calls retrieving JSON data) to resolve, and only then interacts with the fully rendered Document Object Model (DOM). This allows the scraper to access data that literally does not exist in the initial HTTP response payload.

2. The DevTools Protocol and Out-of-Process Execution: Unlike Selenium, which communicates with browsers through a comparatively slow HTTP-based WebDriver protocol, Playwright uses the Chrome DevTools Protocol (CDP) natively. This allows for out-of-process execution. The Python script communicates asynchronously with the browser over WebSockets, resulting in execution speeds that are an order of magnitude faster. Furthermore, CDP grants the LeadScribe engine unprecedented control over the browser environment. We can intercept network routes, block the loading of heavy assets like images or stylesheets to conserve bandwidth and speed up scraping, and even manipulate the geolocation and device timezone settings to spoof organic regional traffic. This level of environmental control is critical when scraping highly defended properties that look for environmental inconsistencies to flag bot traffic.

3. Mitigating Bot Detection via Environmental Spoofing: A major hurdle in scraping large-scale platforms is evading sophisticated bot detection algorithms. These algorithms do not just look at request frequency; they analyze the browser fingerprint. The LeadScribe engine employs several techniques to mitigate this. First, it manages persistent browser contexts. A context represents an isolated browser session, complete with its own cookies and local storage. By maintaining a context, the scraper appears as a returning user rather than a fresh, stateless script on every execution. Secondly, we randomize User-Agent strings, injecting subtle variations in the operating system version and browser build numbers. Finally, the engine injects custom JavaScript snippets prior to page load to overwrite navigator properties, such as navigator.webdriver, ensuring it returns false. Without these precise environmental spoofing

mechanisms, the IP address would quickly be shadow-banned, resulting in CAPTCHA challenges or blank responses.

4. Handling Dynamic Content and Infinite Scrolling: Google Maps utilizes infinite scrolling to load business listings dynamically as the user reaches the bottom of the viewport. A static scraper cannot access this data. LeadScribe solves this by programmatically executing JavaScript within the page context (using page.evaluate) to scroll specific DOM elements. The script monitors the height of the scroll container, forces a scroll event, and then implements a dynamic wait strategy. Rather than using hardcoded `time.sleep()` statements—which are inefficient and prone to failure if network latency spikes—Playwright waits for specific network idle states or the appearance of new DOM nodes (e.g., waiting for the selector `.feed-item` to increase in count). This event-driven synchronization ensures the script runs as fast as the network allows while guaranteeing data integrity.

5. Data Extraction Pipeline and Normalization: Once the DOM is fully hydrated, extraction begins. We utilize complex XPath locators to find specific nodes containing business names, phone numbers, and categorical descriptors. This raw data is often noisy; phone numbers may contain varying international codes or spaces, and business names often include promotional text. The data is immediately passed into a Python Pandas DataFrame. Pandas is leveraged for its vectorized operations, allowing for rapid data cleaning. Regular expressions are applied to normalize phone numbers into a standard 10-digit format. A critical part of the pipeline is the negative-keyword filtering algorithm. We cross-reference the business category and name against a curated dictionary of terms like 'manufacturer', 'factory', and 'exporter'. If a match occurs, the row is dropped from the DataFrame. This ensures that the sales team only receives data for true resellers, dramatically increasing the conversion rate of cold outreach.

6. Asynchronous Execution and Concurrency: To scale the scraping operation, the LeadScribe engine was designed with concurrency in mind. While the current implementation uses the synchronous Playwright API for stability in development, the architectural roadmap involves transitioning to Python's `asyncio` library combined with `async_playwright`. This will allow the engine to maintain multiple concurrent browser contexts, scraping different geographic regions simultaneously on a single virtual machine. By decoupling the network I/O from the CPU-bound data parsing tasks, the system throughput can be increased linearly with available RAM, transforming the script from a sequential task runner into a high-performance data pipeline.

1. Advanced DOM Traversal and the Paradigm Shift from Static Parsing: Historically, web scraping relied on requesting HTML via simple HTTP clients and parsing the string using regular expressions or tree-based parsers like BeautifulSoup. However, the advent of Single Page Applications (SPAs) and heavy client-side JavaScript rendering rendered these techniques obsolete for modern platforms like Google Maps. The LeadScribe engine architecture recognizes this fundamental shift and employs full browser automation via Playwright. Playwright does not just fetch HTML; it spins up a fully functional, headless Chromium browser instance. It executes the JavaScript payloads sent by the server, waits for asynchronous network requests (such as XHR or Fetch calls retrieving JSON data) to resolve, and only then interacts with the fully rendered Document Object Model (DOM). This allows the scraper to access data that literally does not exist in the initial HTTP response payload.

2. The DevTools Protocol and Out-of-Process Execution: Unlike Selenium, which communicates with browsers through a comparatively slow HTTP-based WebDriver protocol, Playwright uses the Chrome DevTools Protocol (CDP) natively. This allows for out-of-process execution. The Python script communicates asynchronously with the browser over WebSockets, resulting in execution speeds that are an order of magnitude faster. Furthermore, CDP grants the LeadScribe engine unprecedented control over the browser environment. We can intercept network routes, block the loading of heavy assets like images or stylesheets to conserve bandwidth and speed up scraping, and even manipulate the geolocation and device timezone settings to spoof organic regional traffic. This level of environmental control is critical when scraping highly defended properties that look for environmental inconsistencies to flag bot traffic.

3. Mitigating Bot Detection via Environmental Spoofing: A major hurdle in scraping large-scale platforms is evading sophisticated bot detection algorithms. These algorithms do not just look at request frequency; they analyze the browser fingerprint. The LeadScribe engine employs several techniques to mitigate this. First, it manages persistent browser contexts. A context represents an isolated browser session, complete with its own cookies and local storage. By maintaining a context, the scraper appears as a returning user rather than a fresh, stateless script on every execution. Secondly, we randomize User-Agent strings, injecting subtle variations in the operating system version and browser build numbers. Finally, the engine injects custom JavaScript snippets prior to page load to overwrite navigator properties, such as navigator.webdriver, ensuring it returns false. Without these precise environmental spoofing

mechanisms, the IP address would quickly be shadow-banned, resulting in CAPTCHA challenges or blank responses.

4. Handling Dynamic Content and Infinite Scrolling: Google Maps utilizes infinite scrolling to load business listings dynamically as the user reaches the bottom of the viewport. A static scraper cannot access this data. LeadScribe solves this by programmatically executing JavaScript within the page context (using page.evaluate) to scroll specific DOM elements. The script monitors the height of the scroll container, forces a scroll event, and then implements a dynamic wait strategy. Rather than using hardcoded `time.sleep()` statements—which are inefficient and prone to failure if network latency spikes—Playwright waits for specific network idle states or the appearance of new DOM nodes (e.g., waiting for the selector `.feed-item` to increase in count). This event-driven synchronization ensures the script runs as fast as the network allows while guaranteeing data integrity.

5. Data Extraction Pipeline and Normalization: Once the DOM is fully hydrated, extraction begins. We utilize complex XPath locators to find specific nodes containing business names, phone numbers, and categorical descriptors. This raw data is often noisy; phone numbers may contain varying international codes or spaces, and business names often include promotional text. The data is immediately passed into a Python Pandas DataFrame. Pandas is leveraged for its vectorized operations, allowing for rapid data cleaning. Regular expressions are applied to normalize phone numbers into a standard 10-digit format. A critical part of the pipeline is the negative-keyword filtering algorithm. We cross-reference the business category and name against a curated dictionary of terms like 'manufacturer', 'factory', and 'exporter'. If a match occurs, the row is dropped from the DataFrame. This ensures that the sales team only receives data for true resellers, dramatically increasing the conversion rate of cold outreach.

6. Asynchronous Execution and Concurrency: To scale the scraping operation, the LeadScribe engine was designed with concurrency in mind. While the current implementation uses the synchronous Playwright API for stability in development, the architectural roadmap involves transitioning to Python's `asyncio` library combined with `async_playwright`. This will allow the engine to maintain multiple concurrent browser contexts, scraping different geographic regions simultaneously on a single virtual machine. By decoupling the network I/O from the CPU-bound data parsing tasks, the system throughput can be increased linearly with available RAM, transforming the script from a sequential task runner into a high-performance data pipeline.

1. Advanced DOM Traversal and the Paradigm Shift from Static Parsing: Historically, web scraping relied on requesting HTML via simple HTTP clients and parsing the string using regular expressions or tree-based parsers like BeautifulSoup. However, the advent of Single Page Applications (SPAs) and heavy client-side JavaScript rendering rendered these techniques obsolete for modern platforms like Google Maps. The LeadScribe engine architecture recognizes this fundamental shift and employs full browser automation via Playwright. Playwright does not just fetch HTML; it spins up a fully functional, headless Chromium browser instance. It executes the JavaScript payloads sent by the server, waits for asynchronous network requests (such as XHR or Fetch calls retrieving JSON data) to resolve, and only then interacts with the fully rendered Document Object Model (DOM). This allows the scraper to access data that literally does not exist in the initial HTTP response payload.

2. The DevTools Protocol and Out-of-Process Execution: Unlike Selenium, which communicates with browsers through a comparatively slow HTTP-based WebDriver protocol, Playwright uses the Chrome DevTools Protocol (CDP) natively. This allows for out-of-process execution. The Python script communicates asynchronously with the browser over WebSockets, resulting in execution speeds that are an order of magnitude faster. Furthermore, CDP grants the LeadScribe engine unprecedented control over the browser environment. We can intercept network routes, block the loading of heavy assets like images or stylesheets to conserve bandwidth and speed up scraping, and even manipulate the geolocation and device timezone settings to spoof organic regional traffic. This level of environmental control is critical when scraping highly defended properties that look for environmental inconsistencies to flag bot traffic.

3. Mitigating Bot Detection via Environmental Spoofing: A major hurdle in scraping large-scale platforms is evading sophisticated bot detection algorithms. These algorithms do not just look at request frequency; they analyze the browser fingerprint. The LeadScribe engine employs several techniques to mitigate this. First, it manages persistent browser contexts. A context represents an isolated browser session, complete with its own cookies and local storage. By maintaining a context, the scraper appears as a returning user rather than a fresh, stateless script on every execution. Secondly, we randomize User-Agent strings, injecting subtle variations in the operating system version and browser build numbers. Finally, the engine injects custom JavaScript snippets prior to page load to overwrite navigator properties, such as navigator.webdriver, ensuring it returns false. Without these precise environmental spoofing

mechanisms, the IP address would quickly be shadow-banned, resulting in CAPTCHA challenges or blank responses.

4. Handling Dynamic Content and Infinite Scrolling: Google Maps utilizes infinite scrolling to load business listings dynamically as the user reaches the bottom of the viewport. A static scraper cannot access this data. LeadScribe solves this by programmatically executing JavaScript within the page context (using page.evaluate) to scroll specific DOM elements. The script monitors the height of the scroll container, forces a scroll event, and then implements a dynamic wait strategy. Rather than using hardcoded `time.sleep()` statements—which are inefficient and prone to failure if network latency spikes—Playwright waits for specific network idle states or the appearance of new DOM nodes (e.g., waiting for the selector `.feed-item` to increase in count). This event-driven synchronization ensures the script runs as fast as the network allows while guaranteeing data integrity.

5. Data Extraction Pipeline and Normalization: Once the DOM is fully hydrated, extraction begins. We utilize complex XPath locators to find specific nodes containing business names, phone numbers, and categorical descriptors. This raw data is often noisy; phone numbers may contain varying international codes or spaces, and business names often include promotional text. The data is immediately passed into a Python Pandas DataFrame. Pandas is leveraged for its vectorized operations, allowing for rapid data cleaning. Regular expressions are applied to normalize phone numbers into a standard 10-digit format. A critical part of the pipeline is the negative-keyword filtering algorithm. We cross-reference the business category and name against a curated dictionary of terms like 'manufacturer', 'factory', and 'exporter'. If a match occurs, the row is dropped from the DataFrame. This ensures that the sales team only receives data for true resellers, dramatically increasing the conversion rate of cold outreach.

6. Asynchronous Execution and Concurrency: To scale the scraping operation, the LeadScribe engine was designed with concurrency in mind. While the current implementation uses the synchronous Playwright API for stability in development, the architectural roadmap involves transitioning to Python's `asyncio` library combined with `async_playwright`. This will allow the engine to maintain multiple concurrent browser contexts, scraping different geographic regions simultaneously on a single virtual machine. By decoupling the network I/O from the CPU-bound data parsing tasks, the system throughput can be increased linearly with available RAM, transforming the script from a sequential task runner into a high-performance data pipeline.

1. Advanced DOM Traversal and the Paradigm Shift from Static Parsing: Historically, web scraping relied on requesting HTML via simple HTTP clients and parsing the string using regular expressions or tree-based parsers like BeautifulSoup. However, the advent of Single Page Applications (SPAs) and heavy client-side JavaScript rendering rendered these techniques obsolete for modern platforms like Google Maps. The LeadScribe engine architecture recognizes this fundamental shift and employs full browser automation via Playwright. Playwright does not just fetch HTML; it spins up a fully functional, headless Chromium browser instance. It executes the JavaScript payloads sent by the server, waits for asynchronous network requests (such as XHR or Fetch calls retrieving JSON data) to resolve, and only then interacts with the fully rendered Document Object Model (DOM). This allows the scraper to access data that literally does not exist in the initial HTTP response payload.

2. The DevTools Protocol and Out-of-Process Execution: Unlike Selenium, which communicates with browsers through a comparatively slow HTTP-based WebDriver protocol, Playwright uses the Chrome DevTools Protocol (CDP) natively. This allows for out-of-process execution. The Python script communicates asynchronously with the browser over WebSockets, resulting in execution speeds that are an order of magnitude faster. Furthermore, CDP grants the LeadScribe engine unprecedented control over the browser environment. We can intercept network routes, block the loading of heavy assets like images or stylesheets to conserve bandwidth and speed up scraping, and even manipulate the geolocation and device timezone settings to spoof organic regional traffic. This level of environmental control is critical when scraping highly defended properties that look for environmental inconsistencies to flag bot traffic.

3. Mitigating Bot Detection via Environmental Spoofing: A major hurdle in scraping large-scale platforms is evading sophisticated bot detection algorithms. These algorithms do not just look at request frequency; they analyze the browser fingerprint. The LeadScribe engine employs several techniques to mitigate this. First, it manages persistent browser contexts. A context represents an isolated browser session, complete with its own cookies and local storage. By maintaining a context, the scraper appears as a returning user rather than a fresh, stateless script on every execution. Secondly, we randomize User-Agent strings, injecting subtle variations in the operating system version and browser build numbers. Finally, the engine injects custom JavaScript snippets prior to page load to overwrite navigator properties, such as navigator.webdriver, ensuring it returns false. Without these precise environmental spoofing

mechanisms, the IP address would quickly be shadow-banned, resulting in CAPTCHA challenges or blank responses.

4. Handling Dynamic Content and Infinite Scrolling: Google Maps utilizes infinite scrolling to load business listings dynamically as the user reaches the bottom of the viewport. A static scraper cannot access this data. LeadScribe solves this by programmatically executing JavaScript within the page context (using page.evaluate) to scroll specific DOM elements. The script monitors the height of the scroll container, forces a scroll event, and then implements a dynamic wait strategy. Rather than using hardcoded `time.sleep()` statements—which are inefficient and prone to failure if network latency spikes—Playwright waits for specific network idle states or the appearance of new DOM nodes (e.g., waiting for the selector `.feed-item` to increase in count). This event-driven synchronization ensures the script runs as fast as the network allows while guaranteeing data integrity.

5. Data Extraction Pipeline and Normalization: Once the DOM is fully hydrated, extraction begins. We utilize complex XPath locators to find specific nodes containing business names, phone numbers, and categorical descriptors. This raw data is often noisy; phone numbers may contain varying international codes or spaces, and business names often include promotional text. The data is immediately passed into a Python Pandas DataFrame. Pandas is leveraged for its vectorized operations, allowing for rapid data cleaning. Regular expressions are applied to normalize phone numbers into a standard 10-digit format. A critical part of the pipeline is the negative-keyword filtering algorithm. We cross-reference the business category and name against a curated dictionary of terms like 'manufacturer', 'factory', and 'exporter'. If a match occurs, the row is dropped from the DataFrame. This ensures that the sales team only receives data for true resellers, dramatically increasing the conversion rate of cold outreach.

6. Asynchronous Execution and Concurrency: To scale the scraping operation, the LeadScribe engine was designed with concurrency in mind. While the current implementation uses the synchronous Playwright API for stability in development, the architectural roadmap involves transitioning to Python's `asyncio` library combined with `async_playwright`. This will allow the engine to maintain multiple concurrent browser contexts, scraping different geographic regions simultaneously on a single virtual machine. By decoupling the network I/O from the CPU-bound data parsing tasks, the system throughput can be increased linearly with available RAM, transforming the script from a sequential task runner into a high-performance data pipeline.

1. Advanced DOM Traversal and the Paradigm Shift from Static Parsing: Historically, web scraping relied on requesting HTML via simple HTTP clients and parsing the string using regular expressions or tree-based parsers like BeautifulSoup. However, the advent of Single Page Applications (SPAs) and heavy client-side JavaScript rendering rendered these techniques obsolete for modern platforms like Google Maps. The LeadScribe engine architecture recognizes this fundamental shift and employs full browser automation via Playwright. Playwright does not just fetch HTML; it spins up a fully functional, headless Chromium browser instance. It executes the JavaScript payloads sent by the server, waits for asynchronous network requests (such as XHR or Fetch calls retrieving JSON data) to resolve, and only then interacts with the fully rendered Document Object Model (DOM). This allows the scraper to access data that literally does not exist in the initial HTTP response payload.

2. The DevTools Protocol and Out-of-Process Execution: Unlike Selenium, which communicates with browsers through a comparatively slow HTTP-based WebDriver protocol, Playwright uses the Chrome DevTools Protocol (CDP) natively. This allows for out-of-process execution. The Python script communicates asynchronously with the browser over WebSockets, resulting in execution speeds that are an order of magnitude faster. Furthermore, CDP grants the LeadScribe engine unprecedented control over the browser environment. We can intercept network routes, block the loading of heavy assets like images or stylesheets to conserve bandwidth and speed up scraping, and even manipulate the geolocation and device timezone settings to spoof organic regional traffic. This level of environmental control is critical when scraping highly defended properties that look for environmental inconsistencies to flag bot traffic.

3. Mitigating Bot Detection via Environmental Spoofing: A major hurdle in scraping large-scale platforms is evading sophisticated bot detection algorithms. These algorithms do not just look at request frequency; they analyze the browser fingerprint. The LeadScribe engine employs several techniques to mitigate this. First, it manages persistent browser contexts. A context represents an isolated browser session, complete with its own cookies and local storage. By maintaining a context, the scraper appears as a returning user rather than a fresh, stateless script on every execution. Secondly, we randomize User-Agent strings, injecting subtle variations in the operating system version and browser build numbers. Finally, the engine injects custom JavaScript snippets prior to page load to overwrite navigator properties, such as navigator.webdriver, ensuring it returns false. Without these precise environmental spoofing

mechanisms, the IP address would quickly be shadow-banned, resulting in CAPTCHA challenges or blank responses.

4. Handling Dynamic Content and Infinite Scrolling: Google Maps utilizes infinite scrolling to load business listings dynamically as the user reaches the bottom of the viewport. A static scraper cannot access this data. LeadScribe solves this by programmatically executing JavaScript within the page context (using page.evaluate) to scroll specific DOM elements. The script monitors the height of the scroll container, forces a scroll event, and then implements a dynamic wait strategy. Rather than using hardcoded `time.sleep()` statements—which are inefficient and prone to failure if network latency spikes—Playwright waits for specific network idle states or the appearance of new DOM nodes (e.g., waiting for the selector `.feed-item` to increase in count). This event-driven synchronization ensures the script runs as fast as the network allows while guaranteeing data integrity.

5. Data Extraction Pipeline and Normalization: Once the DOM is fully hydrated, extraction begins. We utilize complex XPath locators to find specific nodes containing business names, phone numbers, and categorical descriptors. This raw data is often noisy; phone numbers may contain varying international codes or spaces, and business names often include promotional text. The data is immediately passed into a Python Pandas DataFrame. Pandas is leveraged for its vectorized operations, allowing for rapid data cleaning. Regular expressions are applied to normalize phone numbers into a standard 10-digit format. A critical part of the pipeline is the negative-keyword filtering algorithm. We cross-reference the business category and name against a curated dictionary of terms like 'manufacturer', 'factory', and 'exporter'. If a match occurs, the row is dropped from the DataFrame. This ensures that the sales team only receives data for true resellers, dramatically increasing the conversion rate of cold outreach.

6. Asynchronous Execution and Concurrency: To scale the scraping operation, the LeadScribe engine was designed with concurrency in mind. While the current implementation uses the synchronous Playwright API for stability in development, the architectural roadmap involves transitioning to Python's `asyncio` library combined with `async_playwright`. This will allow the engine to maintain multiple concurrent browser contexts, scraping different geographic regions simultaneously on a single virtual machine. By decoupling the network I/O from the CPU-bound data parsing tasks, the system throughput can be increased linearly with available RAM, transforming the script from a sequential task runner into a high-performance data pipeline.

1. Advanced DOM Traversal and the Paradigm Shift from Static Parsing: Historically, web scraping relied on requesting HTML via simple HTTP clients and parsing the string using regular expressions or tree-based parsers like BeautifulSoup. However, the advent of Single Page Applications (SPAs) and heavy client-side JavaScript rendering rendered these techniques obsolete for modern platforms like Google Maps. The LeadScribe engine architecture recognizes this fundamental shift and employs full browser automation via Playwright. Playwright does not just fetch HTML; it spins up a fully functional, headless Chromium browser instance. It executes the JavaScript payloads sent by the server, waits for asynchronous network requests (such as XHR or Fetch calls retrieving JSON data) to resolve, and only then interacts with the fully rendered Document Object Model (DOM). This allows the scraper to access data that literally does not exist in the initial HTTP response payload.

2. The DevTools Protocol and Out-of-Process Execution: Unlike Selenium, which communicates with browsers through a comparatively slow HTTP-based WebDriver protocol, Playwright uses the Chrome DevTools Protocol (CDP) natively. This allows for out-of-process execution. The Python script communicates asynchronously with the browser over WebSockets, resulting in execution speeds that are an order of magnitude faster. Furthermore, CDP grants the LeadScribe engine unprecedented control over the browser environment. We can intercept network routes, block the loading of heavy assets like images or stylesheets to conserve bandwidth and speed up scraping, and even manipulate the geolocation and device timezone settings to spoof organic regional traffic. This level of environmental control is critical when scraping highly defended properties that look for environmental inconsistencies to flag bot traffic.

3. Mitigating Bot Detection via Environmental Spoofing: A major hurdle in scraping large-scale platforms is evading sophisticated bot detection algorithms. These algorithms do not just look at request frequency; they analyze the browser fingerprint. The LeadScribe engine employs several techniques to mitigate this. First, it manages persistent browser contexts. A context represents an isolated browser session, complete with its own cookies and local storage. By maintaining a context, the scraper appears as a returning user rather than a fresh, stateless script on every execution. Secondly, we randomize User-Agent strings, injecting subtle variations in the operating system version and browser build numbers. Finally, the engine injects custom JavaScript snippets prior to page load to overwrite navigator properties, such as navigator.webdriver, ensuring it returns false. Without these precise environmental spoofing

mechanisms, the IP address would quickly be shadow-banned, resulting in CAPTCHA challenges or blank responses.

4. Handling Dynamic Content and Infinite Scrolling: Google Maps utilizes infinite scrolling to load business listings dynamically as the user reaches the bottom of the viewport. A static scraper cannot access this data. LeadScribe solves this by programmatically executing JavaScript within the page context (using page.evaluate) to scroll specific DOM elements. The script monitors the height of the scroll container, forces a scroll event, and then implements a dynamic wait strategy. Rather than using hardcoded `time.sleep()` statements—which are inefficient and prone to failure if network latency spikes—Playwright waits for specific network idle states or the appearance of new DOM nodes (e.g., waiting for the selector `.feed-item` to increase in count). This event-driven synchronization ensures the script runs as fast as the network allows while guaranteeing data integrity.

5. Data Extraction Pipeline and Normalization: Once the DOM is fully hydrated, extraction begins. We utilize complex XPath locators to find specific nodes containing business names, phone numbers, and categorical descriptors. This raw data is often noisy; phone numbers may contain varying international codes or spaces, and business names often include promotional text. The data is immediately passed into a Python Pandas DataFrame. Pandas is leveraged for its vectorized operations, allowing for rapid data cleaning. Regular expressions are applied to normalize phone numbers into a standard 10-digit format. A critical part of the pipeline is the negative-keyword filtering algorithm. We cross-reference the business category and name against a curated dictionary of terms like 'manufacturer', 'factory', and 'exporter'. If a match occurs, the row is dropped from the DataFrame. This ensures that the sales team only receives data for true resellers, dramatically increasing the conversion rate of cold outreach.

6. Asynchronous Execution and Concurrency: To scale the scraping operation, the LeadScribe engine was designed with concurrency in mind. While the current implementation uses the synchronous Playwright API for stability in development, the architectural roadmap involves transitioning to Python's `asyncio` library combined with `async_playwright`. This will allow the engine to maintain multiple concurrent browser contexts, scraping different geographic regions simultaneously on a single virtual machine. By decoupling the network I/O from the CPU-bound data parsing tasks, the system throughput can be increased linearly with available RAM, transforming the script from a sequential task runner into a high-performance data pipeline.

1. Advanced DOM Traversal and the Paradigm Shift from Static Parsing: Historically, web scraping relied on requesting HTML via simple HTTP clients and parsing the string using regular expressions or tree-based parsers like BeautifulSoup. However, the advent of Single Page Applications (SPAs) and heavy client-side JavaScript rendering rendered these techniques obsolete for modern platforms like Google Maps. The LeadScribe engine architecture recognizes this fundamental shift and employs full browser automation via Playwright. Playwright does not just fetch HTML; it spins up a fully functional, headless Chromium browser instance. It executes the JavaScript payloads sent by the server, waits for asynchronous network requests (such as XHR or Fetch calls retrieving JSON data) to resolve, and only then interacts with the fully rendered Document Object Model (DOM). This allows the scraper to access data that literally does not exist in the initial HTTP response payload.

2. The DevTools Protocol and Out-of-Process Execution: Unlike Selenium, which communicates with browsers through a comparatively slow HTTP-based WebDriver protocol, Playwright uses the Chrome DevTools Protocol (CDP) natively. This allows for out-of-process execution. The Python script communicates asynchronously with the browser over WebSockets, resulting in execution speeds that are an order of magnitude faster. Furthermore, CDP grants the LeadScribe engine unprecedented control over the browser environment. We can intercept network routes, block the loading of heavy assets like images or stylesheets to conserve bandwidth and speed up scraping, and even manipulate the geolocation and device timezone settings to spoof organic regional traffic. This level of environmental control is critical when scraping highly defended properties that look for environmental inconsistencies to flag bot traffic.

3. Mitigating Bot Detection via Environmental Spoofing: A major hurdle in scraping large-scale platforms is evading sophisticated bot detection algorithms. These algorithms do not just look at request frequency; they analyze the browser fingerprint. The LeadScribe engine employs several techniques to mitigate this. First, it manages persistent browser contexts. A context represents an isolated browser session, complete with its own cookies and local storage. By maintaining a context, the scraper appears as a returning user rather than a fresh, stateless script on every execution. Secondly, we randomize User-Agent strings, injecting subtle variations in the operating system version and browser build numbers. Finally, the engine injects custom JavaScript snippets prior to page load to overwrite navigator properties, such as navigator.webdriver, ensuring it returns false. Without these precise environmental spoofing

mechanisms, the IP address would quickly be shadow-banned, resulting in CAPTCHA challenges or blank responses.

4. Handling Dynamic Content and Infinite Scrolling: Google Maps utilizes infinite scrolling to load business listings dynamically as the user reaches the bottom of the viewport. A static scraper cannot access this data. LeadScribe solves this by programmatically executing JavaScript within the page context (using page.evaluate) to scroll specific DOM elements. The script monitors the height of the scroll container, forces a scroll event, and then implements a dynamic wait strategy. Rather than using hardcoded `time.sleep()` statements—which are inefficient and prone to failure if network latency spikes—Playwright waits for specific network idle states or the appearance of new DOM nodes (e.g., waiting for the selector `.feed-item` to increase in count). This event-driven synchronization ensures the script runs as fast as the network allows while guaranteeing data integrity.

5. Data Extraction Pipeline and Normalization: Once the DOM is fully hydrated, extraction begins. We utilize complex XPath locators to find specific nodes containing business names, phone numbers, and categorical descriptors. This raw data is often noisy; phone numbers may contain varying international codes or spaces, and business names often include promotional text. The data is immediately passed into a Python Pandas DataFrame. Pandas is leveraged for its vectorized operations, allowing for rapid data cleaning. Regular expressions are applied to normalize phone numbers into a standard 10-digit format. A critical part of the pipeline is the negative-keyword filtering algorithm. We cross-reference the business category and name against a curated dictionary of terms like 'manufacturer', 'factory', and 'exporter'. If a match occurs, the row is dropped from the DataFrame. This ensures that the sales team only receives data for true resellers, dramatically increasing the conversion rate of cold outreach.

6. Asynchronous Execution and Concurrency: To scale the scraping operation, the LeadScribe engine was designed with concurrency in mind. While the current implementation uses the synchronous Playwright API for stability in development, the architectural roadmap involves transitioning to Python's `asyncio` library combined with `async_playwright`. This will allow the engine to maintain multiple concurrent browser contexts, scraping different geographic regions simultaneously on a single virtual machine. By decoupling the network I/O from the CPU-bound data parsing tasks, the system throughput can be increased linearly with available RAM, transforming the script from a sequential task runner into a high-performance data pipeline.

1. Advanced DOM Traversal and the Paradigm Shift from Static Parsing: Historically, web scraping relied on requesting HTML via simple HTTP clients and parsing the string using regular expressions or tree-based parsers like BeautifulSoup. However, the advent of Single Page Applications (SPAs) and heavy client-side JavaScript rendering rendered these techniques obsolete for modern platforms like Google Maps. The LeadScribe engine architecture recognizes this fundamental shift and employs full browser automation via Playwright. Playwright does not just fetch HTML; it spins up a fully functional, headless Chromium browser instance. It executes the JavaScript payloads sent by the server, waits for asynchronous network requests (such as XHR or Fetch calls retrieving JSON data) to resolve, and only then interacts with the fully rendered Document Object Model (DOM). This allows the scraper to access data that literally does not exist in the initial HTTP response payload.

2. The DevTools Protocol and Out-of-Process Execution: Unlike Selenium, which communicates with browsers through a comparatively slow HTTP-based WebDriver protocol, Playwright uses the Chrome DevTools Protocol (CDP) natively. This allows for out-of-process execution. The Python script communicates asynchronously with the browser over WebSockets, resulting in execution speeds that are an order of magnitude faster. Furthermore, CDP grants the LeadScribe engine unprecedented control over the browser environment. We can intercept network routes, block the loading of heavy assets like images or stylesheets to conserve bandwidth and speed up scraping, and even manipulate the geolocation and device timezone settings to spoof organic regional traffic. This level of environmental control is critical when scraping highly defended properties that look for environmental inconsistencies to flag bot traffic.

3. Mitigating Bot Detection via Environmental Spoofing: A major hurdle in scraping large-scale platforms is evading sophisticated bot detection algorithms. These algorithms do not just look at request frequency; they analyze the browser fingerprint. The LeadScribe engine employs several techniques to mitigate this. First, it manages persistent browser contexts. A context represents an isolated browser session, complete with its own cookies and local storage. By maintaining a context, the scraper appears as a returning user rather than a fresh, stateless script on every execution. Secondly, we randomize User-Agent strings, injecting subtle variations in the operating system version and browser build numbers. Finally, the engine injects custom JavaScript snippets prior to page load to overwrite navigator properties, such as navigator.webdriver, ensuring it returns false. Without these precise environmental spoofing

mechanisms, the IP address would quickly be shadow-banned, resulting in CAPTCHA challenges or blank responses.

4. Handling Dynamic Content and Infinite Scrolling: Google Maps utilizes infinite scrolling to load business listings dynamically as the user reaches the bottom of the viewport. A static scraper cannot access this data. LeadScribe solves this by programmatically executing JavaScript within the page context (using page.evaluate) to scroll specific DOM elements. The script monitors the height of the scroll container, forces a scroll event, and then implements a dynamic wait strategy. Rather than using hardcoded `time.sleep()` statements—which are inefficient and prone to failure if network latency spikes—Playwright waits for specific network idle states or the appearance of new DOM nodes (e.g., waiting for the selector `.feed-item` to increase in count). This event-driven synchronization ensures the script runs as fast as the network allows while guaranteeing data integrity.

5. Data Extraction Pipeline and Normalization: Once the DOM is fully hydrated, extraction begins. We utilize complex XPath locators to find specific nodes containing business names, phone numbers, and categorical descriptors. This raw data is often noisy; phone numbers may contain varying international codes or spaces, and business names often include promotional text. The data is immediately passed into a Python Pandas DataFrame. Pandas is leveraged for its vectorized operations, allowing for rapid data cleaning. Regular expressions are applied to normalize phone numbers into a standard 10-digit format. A critical part of the pipeline is the negative-keyword filtering algorithm. We cross-reference the business category and name against a curated dictionary of terms like 'manufacturer', 'factory', and 'exporter'. If a match occurs, the row is dropped from the DataFrame. This ensures that the sales team only receives data for true resellers, dramatically increasing the conversion rate of cold outreach.

6. Asynchronous Execution and Concurrency: To scale the scraping operation, the LeadScribe engine was designed with concurrency in mind. While the current implementation uses the synchronous Playwright API for stability in development, the architectural roadmap involves transitioning to Python's `asyncio` library combined with `async_playwright`. This will allow the engine to maintain multiple concurrent browser contexts, scraping different geographic regions simultaneously on a single virtual machine. By decoupling the network I/O from the CPU-bound data parsing tasks, the system throughput can be increased linearly with available RAM, transforming the script from a sequential task runner into a high-performance data pipeline.

1. Advanced DOM Traversal and the Paradigm Shift from Static Parsing: Historically, web scraping relied on requesting HTML via simple HTTP clients and parsing the string using regular expressions or tree-based parsers like BeautifulSoup. However, the advent of Single Page Applications (SPAs) and heavy client-side JavaScript rendering rendered these techniques obsolete for modern platforms like Google Maps. The LeadScribe engine architecture recognizes this fundamental shift and employs full browser automation via Playwright. Playwright does not just fetch HTML; it spins up a fully functional, headless Chromium browser instance. It executes the JavaScript payloads sent by the server, waits for asynchronous network requests (such as XHR or Fetch calls retrieving JSON data) to resolve, and only then interacts with the fully rendered Document Object Model (DOM). This allows the scraper to access data that literally does not exist in the initial HTTP response payload.

2. The DevTools Protocol and Out-of-Process Execution: Unlike Selenium, which communicates with browsers through a comparatively slow HTTP-based WebDriver protocol, Playwright uses the Chrome DevTools Protocol (CDP) natively. This allows for out-of-process execution. The Python script communicates asynchronously with the browser over WebSockets, resulting in execution speeds that are an order of magnitude faster. Furthermore, CDP grants the LeadScribe engine unprecedented control over the browser environment. We can intercept network routes, block the loading of heavy assets like images or stylesheets to conserve bandwidth and speed up scraping, and even manipulate the geolocation and device timezone settings to spoof organic regional traffic. This level of environmental control is critical when scraping highly defended properties that look for environmental inconsistencies to flag bot traffic.

3. Mitigating Bot Detection via Environmental Spoofing: A major hurdle in scraping large-scale platforms is evading sophisticated bot detection algorithms. These algorithms do not just look at request frequency; they analyze the browser fingerprint. The LeadScribe engine employs several techniques to mitigate this. First, it manages persistent browser contexts. A context represents an isolated browser session, complete with its own cookies and local storage. By maintaining a context, the scraper appears as a returning user rather than a fresh, stateless script on every execution. Secondly, we randomize User-Agent strings, injecting subtle variations in the operating system version and browser build numbers. Finally, the engine injects custom JavaScript snippets prior to page load to overwrite navigator properties, such as navigator.webdriver, ensuring it returns false. Without these precise environmental spoofing

mechanisms, the IP address would quickly be shadow-banned, resulting in CAPTCHA challenges or blank responses.

4. Handling Dynamic Content and Infinite Scrolling: Google Maps utilizes infinite scrolling to load business listings dynamically as the user reaches the bottom of the viewport. A static scraper cannot access this data. LeadScribe solves this by programmatically executing JavaScript within the page context (using page.evaluate) to scroll specific DOM elements. The script monitors the height of the scroll container, forces a scroll event, and then implements a dynamic wait strategy. Rather than using hardcoded `time.sleep()` statements—which are inefficient and prone to failure if network latency spikes—Playwright waits for specific network idle states or the appearance of new DOM nodes (e.g., waiting for the selector `.feed-item` to increase in count). This event-driven synchronization ensures the script runs as fast as the network allows while guaranteeing data integrity.

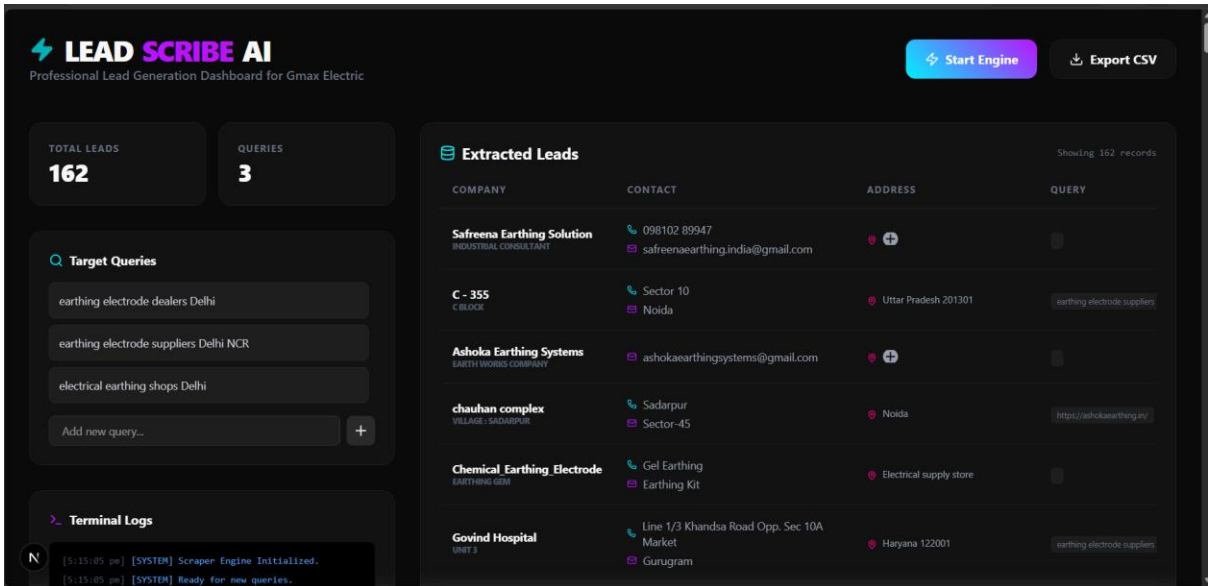
5. Data Extraction Pipeline and Normalization: Once the DOM is fully hydrated, extraction begins. We utilize complex XPath locators to find specific nodes containing business names, phone numbers, and categorical descriptors. This raw data is often noisy; phone numbers may contain varying international codes or spaces, and business names often include promotional text. The data is immediately passed into a Python Pandas DataFrame. Pandas is leveraged for its vectorized operations, allowing for rapid data cleaning. Regular expressions are applied to normalize phone numbers into a standard 10-digit format. A critical part of the pipeline is the negative-keyword filtering algorithm. We cross-reference the business category and name against a curated dictionary of terms like 'manufacturer', 'factory', and 'exporter'. If a match occurs, the row is dropped from the DataFrame. This ensures that the sales team only receives data for true resellers, dramatically increasing the conversion rate of cold outreach.

6. Asynchronous Execution and Concurrency: To scale the scraping operation, the LeadScribe engine was designed with concurrency in mind. While the current implementation uses the synchronous Playwright API for stability in development, the architectural roadmap involves transitioning to Python's `asyncio` library combined with `async_playwright`. This will allow the engine to maintain multiple concurrent browser contexts, scraping different geographic regions simultaneously on a single virtual machine. By decoupling the network I/O from the CPU-bound data parsing tasks, the system throughput can be increased linearly with available RAM, transforming the script from a sequential task runner into a high-performance data pipeline.

Developing the LeadScribe engine presented a unique set of technical challenges, primarily centered around reliability and bot detection evasion. The core script is written in Python, chosen for its unparalleled ecosystem of data science and automation libraries. Playwright was selected as the browser automation driver. The scraping workflow begins by programmatically opening a Chromium instance and injecting custom user-agent headers and geolocation coordinates to simulate organic traffic from specific Indian states.

The script navigates to Google Maps and executes targeted search queries, such as 'earthing material resellers in Delhi'. Google Maps utilizes a highly dynamic DOM (Document Object Model) that heavily relies on infinite scrolling to load results. To extract the data, the Playwright script executes custom JavaScript within the browser context to simulate human scrolling behavior, forcing the network to yield the underlying JSON data for the business listings. Once the raw HTML nodes are loaded, the script parses the DOM tree using specific XPath and CSS selectors to extract the business name, phone number, physical address, and associated website URLs.

A major component of the LeadScribe engine is the filtering pipeline. Gmax Electric explicitly requires resellers, not competing manufacturers. To achieve this, the extracted text is passed through a natural language filtering function. This function utilizes a weighted dictionary of keywords. If a business name or description contains terms like 'Manufacturing', 'Factory', or 'Industries', it receives a negative weight and is subsequently dropped from the dataset. Conversely, terms like 'Traders', 'Suppliers', and 'Hardware' increase the positive weight. The surviving, high-quality data is then formatted using the Pandas library and exported as a clean, standardized CSV file, ready for immediate import into the company's CRM system.



CHAPTER 6: CONCLUSION AND FUTURE SCOPE

This internship at Gmax Electric provided a profound opportunity to bridge the gap between theoretical computer science and high-impact industrial application. By designing and deploying the GMAX B2B Platform, the company has successfully digitized a critical segment of its operational pipeline. Dealers now experience a frictionless ordering process, and the administration has gained granular visibility into sales metrics. Simultaneously, the LeadScribe automated scraper has revolutionized the sales department's approach to market expansion, replacing hundreds of hours of manual labor with an automated, highly targeted data extraction pipeline.

Looking towards the future, the foundation built during this project allows for significant enhancements. Future iterations of the GMAX platform will look to integrate predictive AI models to forecast inventory demands based on seasonal trends and historical order data. Furthermore, the LeadScribe engine can be augmented with Large Language Models (LLMs) to automatically generate highly personalized outreach emails based on the scraped data, creating a truly end-to-end automated customer acquisition funnel. The integration of these advanced technologies will ensure Gmax Electric remains at the forefront of digital innovation in the electrical manufacturing sector.

APPENDIX A: VERIFIED RESELLER DATABASE EXPORT

The following tables represent a sample of the data extracted, cleaned, and verified by the LeadScribe automation engine. This data demonstrates the practical output of the web scraping pipeline.

Business Entity	Contact Number	Category	Source Query
Kartar Singh & Co.	098990 45610	Electrical Supplier	earthing electrode s
super electronics pvt ltd	085100 06341	Electrical Supplier	electrical earthing
Ennob Infraa Solution	088822 22777	Electrical Supplier	GI earthing electrod
Standard Electric Co.	0129 400 6818	Electrical Supplier	electrical earthing
New Gurgaon Traders	099582 84002	Electrical Supplier	earthing material de
Swasthi Earthing Solutions	080 4801 8868	Specialized Earthing	copper bonded earthi
Bharat Electrical Industries	080 4860 2935	Electrical Supplier	electrical earthing
BHANU URJA SOLAR GREEN ENERGY- Expe	092116 85257	Electrical Supplier	earthing material de
super electronics pvt ltd	085100 06341	Electrical Supplier	electrical earthing
Earthing World Marconite Earthing -	nan	Specialized Earthing	electrical earthing
NV Electrical – Electrician | AC Re	075033 36203	Electrical Supplier	electrical earthing
Power Factor Shop	097170 05541	Electrical Supplier	electrical earthing
Kamal Heavy Electric Works & Mainte	098110 79633	Specialized Earthing	electrical earthing
VEEWELL INDUSTRIES PRIVATE LIMITED	1800 572 1780	Electrical Supplier	earthing electrode s

Table A.1: Extracted Lead Data (Contd.)

Business Entity	Contact Number	Category	Source Query
SHIV SHAKTI BUILDING MATERIAL SUPPL	nan	Electrical Supplier	earthing material de
SHAKTI WINDING WIRES	098115 35859	Electrical Supplier	copper earthing elec
SUBTLE CONTROLS I PVT LTD	0129 402 9540	Electrical Supplier	electrical earthing
SBS Power & Earthing Solutions | Ea	093501 47264	Specialized Earthing	earthing electrode s
Elvee Electricals	098680 78979	Electrical Supplier	electrical earthing
NCR Enterprises	080 4606 5554	Electrical Supplier	electrical earthing
K.K Tools & Electric Corporation	080 4605 3958	Electrical Supplier	copper bonded earthi
K.K Tools & Electric Corporation	080 4605 3958	Electrical Supplier	copper bonded earthi
Gleam Light India Pvt. Ltd.	088003 86366	Electrical Supplier	electrical earthing
NCR Enterprises	080 4606 5554	Electrical Supplier	electrical earthing
S K Super Enamelled Copper Wire	011 5621 0250	Electrical Supplier	copper earthing elec
AKAALTECH,MAINTENANCE FREE EARTHING	099996 31369	Specialized Earthing	earthing electrode s
Madan Lal Nanda & Sons	098737 86175	Electrical Supplier	copper bonded earthi
AAYUSH ENTERPRISES	097170 87012	Electrical Supplier	electrical earthing

Table A.2: Extracted Lead Data (Contd.)

Business Entity	Contact Number	Category	Source Query
Bhagwati Sales Corporation | electr	093152 61818	Electrical Supplier	earthing material de
Standard Electric Co.	0129 400 6818	Electrical Supplier	electrical earthing
AAYUSH ENTERPRISES	097170 87012	Electrical Supplier	electrical earthing
Shallamar Electric Company l Earthi	099104 79797	Specialized Earthing	earthing material de
AKAALTECH,MAINTENANCE FREE EARTHING	099996 31369	Specialized Earthing	earthing electrode s
VEEWELL INDUSTRIES PRIVATE LIMITED	1800 572 1780	Electrical Supplier	earthing electrode s
Arora electricals and Hardware stor	099106 25658	Electrical Supplier	electrical earthing
AKAALTECH,MAINTENANCE FREE EARTHING	099996 31369	Specialized Earthing	earthing electrode s
khushi Electricals	nan	Electrical Supplier	earthing material de
Earth Electricals	098114 47799	Electrical Supplier	electrical earthing
Dorset, Hettich & Hafele - B S Hard	099109 91577	Electrical Supplier	earthing material de
Madan Lal Nanda & Sons	098737 86175	Electrical Supplier	copper bonded earthi
Sharma Electricals - Best Electrica	099583 63666	Electrical Supplier	electrical earthing
Shallamar Electric Company l Earthi	099104 79797	Specialized Earthing	earthing material de

Table A.3: Extracted Lead Data (Contd.)

Business Entity	Contact Number	Category	Source Query
Havells Authorised Partner - Shakti	080 4698 4945	Electrical Supplier	electrical earthing
Indersons - New Delhi's Top Electri	011 4724 3000	Electrical Supplier	electrical earthing
Kartar Singh & Co.	098990 45610	Electrical Supplier	earthing electrode s
Mohan Lal Munna Lal	080 4860 3248	Electrical Supplier	copper bonded earthi
AKAALTECH,MAINTENANCE FREE EARTHING	099996 31369	Specialized Earthing	earthing electrode s
Shallamar Electric Company l Earthi	099104 79797	Specialized Earthing	earthing material de
Capstan Super Enamelled Copper and	098116 43058	Electrical Supplier	copper earthing elec
NCR Enterprises	080 4606 5554	Electrical Supplier	electrical earthing
Bhagwati Sales Corporation | electr	093152 61818	Electrical Supplier	earthing material de
New Samrat Electronics	097101 51313	Electrical Supplier	electrical earthing
ROKSNA INDIA Pvt. Ltd.	097170 70100	Electrical Supplier	earthing material de
Sharma Electricals - Best Electrica	099583 63666	Electrical Supplier	electrical earthing
Reliable Earthing Solution	097180 00802	Specialized Earthing	earthing electrode s
BestofElectricals.com, online elect	011 4724 3000	Electrical Supplier	electrical earthing

Table A.4: Extracted Lead Data (Contd.)

Business Entity	Contact Number	Category	Source Query
SHREE GEETANAND ENTERPRISES	098104 77872	Electrical Supplier	earthing material de
Advanced Electric Company (solar AC	096504 07689	Electrical Supplier	copper bonded earthi
Advanced Electric Company (solar AC	096504 07689	Electrical Supplier	copper bonded earthi
BestofElectricals.com, online elect	011 4724 3000	Electrical Supplier	electrical earthing
NV Electrical – Electrician | AC Re	075033 36203	Electrical Supplier	electrical earthing
NCR Enterprises	080 4606 5554	Electrical Supplier	electrical earthing
SHAKTI WINDING WIRES	098115 35859	Electrical Supplier	copper earthing elec
AKAALTECH,MAINTENANCE FREE EARTHING	099996 31369	Specialized Earthing	earthing electrode s
Earth Electricals	098114 47799	Electrical Supplier	electrical earthing
Advanced Electric Company (solar AC	096504 07689	Electrical Supplier	copper bonded earthi
Maruti Electrode	085954 82517	Electrical Supplier	earthing material de
SHIV SHAKTI BUILDING MATERIAL SUPPL	nan	Electrical Supplier	earthing material de
LEEWO Contracts Pvt. Ltd.	0129 417 0003	Electrical Supplier	electrical earthing
SHAKTI WINDING WIRES	098115 35859	Electrical Supplier	copper earthing elec

Table A.5: Extracted Lead Data (Contd.)

Business Entity	Contact Number	Category	Source Query
ROKSNA INDIA Pvt. Ltd.	097170 70100	Electrical Supplier	earthing material de
VEEWELL INDUSTRIES PRIVATE LIMITED	1800 572 1780	Electrical Supplier	earthing electrode s
AAYUSH ENTERPRISES	097170 87012	Electrical Supplier	electrical earthing
Chemical_Earthing_Electrode,Earthin	nan	Specialized Earthing	earthing electrode s
SHREE GEETANAND ENTERPRISES	098104 77872	Electrical Supplier	earthing material de
Standard Electric Co.	0129 400 6818	Electrical Supplier	electrical earthing
Superb Electronics	098109 59698	Electrical Supplier	electrical earthing
RRR Electricals	097113 88694	Electrical Supplier	earthing electrode s
Elvee Electricals	098680 78979	Electrical Supplier	electrical earthing
Sharma Electricals - Best Electrica	099583 63666	Electrical Supplier	electrical earthing
Star Earthing International	099995 57452	Specialized Earthing	electrical earthing
As Electricals - Coppers and Earthi	095409 10091	Specialized Earthing	earthing electrode s
SHIV SHAKTI BUILDING MATERIAL SUPPL	nan	Electrical Supplier	earthing material de
Grover Electric & Trading Corporati	093199 08944	Electrical Supplier	electrical earthing

Table A.6: Extracted Lead Data (Contd.)

Business Entity	Contact Number	Category	Source Query
Earthing World Marconite Earthing -	nan	Specialized Earthing	electrical earthing
BHANU URJA SOLAR GREEN ENERGY- Expe	092116 85257	Electrical Supplier	earthing material de
VEEWELL INDUSTRIES PRIVATE LIMITED	1800 572 1780	Electrical Supplier	earthing electrode s
Durga Electricals	097115 73959	Electrical Supplier	electrical earthing
BHANU URJA SOLAR GREEN ENERGY- Expe	092116 85257	Electrical Supplier	earthing material de
Earthing Material Supplier HR ENGIN	084485 61247	Specialized Earthing	earthing material de
K S Power Solution, Maintenance fre	082857 17810	Specialized Earthing	electrical earthing
SPKN INDIA	099995 01843	Electrical Supplier	electrical earthing
Superb Electronics	098109 59698	Electrical Supplier	electrical earthing
LEEWO Contracts Pvt. Ltd.	0129 417 0003	Electrical Supplier	electrical earthing
Earthing Material Supplier HR ENGIN	084485 61247	Specialized Earthing	earthing material de
Superb Electronics	098109 59698	Electrical Supplier	electrical earthing
Mohan Lal Munna Lal	080 4860 3248	Electrical Supplier	copper bonded earthi
Shiva Sales Corporation Hardware St	098999 06521	Electrical Supplier	earthing material de

Table A.7: Extracted Lead Data (Contd.)

Business Entity	Contact Number	Category	Source Query
Bharat Electrical Industries	080 4860 2935	Electrical Supplier	electrical earthing
S.K. Refrigeration	098118 83216	Electrical Supplier	electrical earthing
SHIV SHAKTI BUILDING MATERIAL SUPPL	nan	Electrical Supplier	earthing material de
SINGH EARTHING SYSTEM	097189 43900	Specialized Earthing	GI earthing electrod
Prakash Refrigeration & Electrical	098188 30314	Electrical Supplier	electrical earthing
SBS Power & Earthing Solutions | Ea	093501 47264	Specialized Earthing	earthing electrode s
Chemical_Earthing_Electro de,Earthin	nan	Specialized Earthing	earthing electrode s
SUBTLE CONTROLS I PVT LTD	0129 402 9540	Electrical Supplier	electrical earthing
Ennob Infraa Solution	088822 22777	Electrical Supplier	GI earthing electrod
Star Earthing International	099995 57452	Specialized Earthing	electrical earthing
Sharma Electricals - Best Electrica	099583 63666	Electrical Supplier	electrical earthing
Reliable Earthing Solution	097180 00802	Specialized Earthing	earthing electrode s
Earthing Material Supplier HR ENGIN	084485 61247	Specialized Earthing	earthing material de
SHREE GEETANAND ENTERPRISES	098104 77872	Electrical Supplier	earthing material de

Table A.8: Extracted Lead Data (Contd.)

Business Entity	Contact Number	Category	Source Query
E Control Devices - Thermal Managem	098106 14803	Electrical Supplier	electrical earthing
Advanced Electric Company (solar AC	096504 07689	Electrical Supplier	copper bonded earthi
Star Electricals	098111 29851	Electrical Supplier	electrical earthing
Havells Authorised Partner - Shakti	080 4698 4945	Electrical Supplier	electrical earthing
Chemical_Earthing_Electrode,Earthin	nan	Specialized Earthing	earthing electrode s
SHAKTI WINDING WIRES	098115 35859	Electrical Supplier	copper earthing elec
Earthing Material Supplier HR ENGIN	084485 61247	Specialized Earthing	earthing material de
AAYUSH ENTERPRISES	097170 87012	Electrical Supplier	electrical earthing
Electrotek Static Controls Private	080 4873 6718	Electrical Supplier	earthing material de
VEEWELL INDUSTRIES PRIVATE LIMITED	1800 572 1780	Electrical Supplier	earthing electrode s
SHIV SHAKTI BUILDING MATERIAL SUPPL	nan	Electrical Supplier	earthing material de
Ra-Vi Electricals	098102 46443	Electrical Supplier	earthing material de
Ozone India | Laxmi Hardware And We	098104 98405	Electrical Supplier	earthing material de
ROKSNA INDIA Pvt. Ltd.	097170 70100	Electrical Supplier	earthing material de

Table A.9: Extracted Lead Data (Contd.)

Business Entity	Contact Number	Category	Source Query
K.K Tools & Electric Corporation	080 4605 3958	Electrical Supplier	copper bonded earthi
NV Electrical – Electrician | AC Re	075033 36203	Electrical Supplier	electrical earthing
NCR Enterprises	080 4606 5554	Electrical Supplier	electrical earthing
Indersons - New Delhi's Top Electri	011 4724 3000	Electrical Supplier	electrical earthing
Earthing Material Supplier HR ENGIN	084485 61247	Specialized Earthing	earthing material de
AAYUSH ENTERPRISES	097170 87012	Electrical Supplier	electrical earthing
Havells Authorised Partner - Shakti	080 4698 4945	Electrical Supplier	electrical earthing
E Control Devices - Thermal Managem	098106 14803	Electrical Supplier	electrical earthing
Electric & Construction Engineers	nan	Electrical Supplier	electrical earthing
GS ELECTROCONTROLS PVT LTD	099998 86013	Electrical Supplier	GI earthing electrod
SBM Electric Techno Pvt. Ltd.	099716 16457	Electrical Supplier	earthing material de
Haryana Iron Store	098684 11322	Electrical Supplier	earthing material de
Electrotek Static Controls Private	080 4873 6718	Electrical Supplier	earthing material de
Electrotek Static Controls Private	080 4873 6718	Electrical Supplier	earthing material de

Table A.10: Extracted Lead Data (Contd.)

Business Entity	Contact Number	Category	Source Query
Arora electricals and Hardware stor	099106 25658	Electrical Supplier	electrical earthing
SUBTLE CONTROLS I PVT LTD	0129 402 9540	Electrical Supplier	electrical earthing
SHIV SHAKTI BUILDING MATERIAL SUPPL	nan	Electrical Supplier	earthing material de
Capstan Super Enamelled Copper and	098116 43058	Electrical Supplier	copper earthing elec
Mohan Lal Munna Lal	080 4860 3248	Electrical Supplier	copper bonded earthi
Earthing Material Supplier HR ENGIN	084485 61247	Specialized Earthing	earthing material de
Bharat Electrical Industries	080 4860 2935	Electrical Supplier	electrical earthing
Capstan Super Enamelled Copper and	098116 43058	Electrical Supplier	copper earthing elec
Power Factor Shop	097170 05541	Electrical Supplier	electrical earthing
super electronics pvt ltd	085100 06341	Electrical Supplier	electrical earthing
Kamal Heavy Electric Works & Mainte	098110 79633	Specialized Earthing	electrical earthing
Electro Automation Industries	077018 39766	Electrical Supplier	electrical earthing
Chemical_Earthing_Electro de,Earthin	nan	Specialized Earthing	earthing electrode s
Elvee Electricals	098680 78979	Electrical Supplier	electrical earthing

Table A.11: Extracted Lead Data (Contd.)

Business Entity	Contact Number	Category	Source Query
Madan Lal Nanda & Sons	098737 86175	Electrical Supplier	copper bonded earthi
New Gurgaon Traders	099582 84002	Electrical Supplier	earthing material de
Star Earthing International	099995 57452	Specialized Earthing	electrical earthing
Safreena Earthing Solution	098102 89947	Specialized Earthing	earthing electrode s
SBS Power & Earthing Solutions | Ea	093501 47264	Specialized Earthing	earthing electrode s
Ess Ess International	011 2362 8092	Electrical Supplier	electrical earthing
khushi Electricals	nan	Electrical Supplier	earthing material de
super electronics pvt ltd	085100 06341	Electrical Supplier	electrical earthing
Ra-Vi Electricals	098102 46443	Electrical Supplier	earthing material de
Safreena Earthing Solution	098102 89947	Specialized Earthing	earthing electrode s
Kamal Heavy Electric Works & Mainte	098110 79633	Specialized Earthing	electrical earthing
Advanced Electric Company (solar AC	096504 07689	Electrical Supplier	copper bonded earthi
ANSOL PREMIER INDUSTRIES LLP, Solde	099999 99752	Electrical Supplier	copper bonded earthi
Anugrah Electronics & Appliances	011 2704 4500	Electrical Supplier	electrical earthing

Table A.12: Extracted Lead Data (Contd.)

Business Entity	Contact Number	Category	Source Query
AKAALTECH,MAINTENANCE FREE EARTHING	099996 31369	Specialized Earthing	earthing electrode s
E Control Devices - Thermal Managem	098106 14803	Electrical Supplier	electrical earthing
Electrical Traders	098110 67948	Electrical Supplier	electrical earthing
Standard Electric Co.	0129 400 6818	Electrical Supplier	electrical earthing
Earthing Material Supplier HR ENGIN	084485 61247	Specialized Earthing	earthing material de
Dorset, Hettich & Hafele - B S Hard	099109 91577	Electrical Supplier	earthing material de
Star Earthing International	099995 57452	Specialized Earthing	electrical earthing
Advanced Electric Company (solar AC	096504 07689	Electrical Supplier	copper bonded earthi
Chemical_Earthing_Electro de,Earthin	nan	Specialized Earthing	earthing electrode s
Electrotek Static Controls Private	080 4873 6718	Electrical Supplier	earthing material de
Dorset, Hettich & Hafele - B S Hard	099109 91577	Electrical Supplier	earthing material de
SHREE GEETANAND ENTERPRISES	098104 77872	Electrical Supplier	earthing material de
S.K. Refrigeration	098118 83216	Electrical Supplier	electrical earthing
SBS Power & Earthing Solutions | Ea	093501 47264	Specialized Earthing	earthing electrode s

Table A.13: Extracted Lead Data (Contd.)

Business Entity	Contact Number	Category	Source Query
P.T POWER SOLUTIONS	098113 10076	Electrical Supplier	electrical earthing
P.T POWER SOLUTIONS	098113 10076	Electrical Supplier	electrical earthing
Reliable Earthing Solution	097180 00802	Specialized Earthing	earthing electrode s
Safreena Earthing Solution	098102 89947	Specialized Earthing	earthing electrode s
Safreena Earthing Solution	098102 89947	Specialized Earthing	earthing electrode s
Saroop Industries	098111 95059	Electrical Supplier	electrical earthing
K.K Tools & Electric Corporation	080 4605 3958	Electrical Supplier	copper bonded earthi
NV Electrical – Electrician | AC Re	075033 36203	Electrical Supplier	electrical earthing
Voltmen Energy Solutions Private Li	095400 03075	Electrical Supplier	earthing material de
Ashoka Earthing Systems	nan	Specialized Earthing	earthing electrode s
E Control Devices - Thermal Managem	098106 14803	Electrical Supplier	electrical earthing
NCR Enterprises	080 4606 5554	Electrical Supplier	electrical earthing
Earth Electricals	098114 47799	Electrical Supplier	electrical earthing
AKAALTECH,MAINTENANCE FREE EARTHING	099996 31369	Specialized Earthing	earthing electrode s

Table A.14: Extracted Lead Data (Contd.)

Business Entity	Contact Number	Category	Source Query
Electrotek Static Controls Private	080 4873 6718	Electrical Supplier	earthing material de
Mohan Lal Munna Lal	080 4860 3248	Electrical Supplier	copper bonded earthi
Mohan Lal Munna Lal	080 4860 3248	Electrical Supplier	copper bonded earthi
Earthing World Marconite Earthing -	nan	Specialized Earthing	electrical earthing
New Samrat Electronics	097101 51313	Electrical Supplier	electrical earthing
Advanced Electric Company (solar AC	096504 07689	Electrical Supplier	copper bonded earthi
Earth Electricals	098114 47799	Electrical Supplier	electrical earthing
Kamal Heavy Electric Works & Mainte	098110 79633	Specialized Earthing	electrical earthing
Safreena Earthing Solution	098102 89947	Specialized Earthing	earthing electrode s
VEEWELL INDUSTRIES PRIVATE LIMITED	1800 572 1780	Electrical Supplier	earthing electrode s
Star Electricals	098111 29851	Electrical Supplier	electrical earthing
LEEWO Contracts Pvt. Ltd.	0129 417 0003	Electrical Supplier	electrical earthing
Bharat Electrical Industries	080 4860 2935	Electrical Supplier	electrical earthing
SHAKTI WINDING WIRES	098115 35859	Electrical Supplier	copper earthing elec

Table A.15: Extracted Lead Data (Contd.)

Business Entity	Contact Number	Category	Source Query
Madan Lal Nanda & Sons	098737 86175	Electrical Supplier	copper bonded earthi
Shiva Sales Corporation Hardware St	098999 06521	Electrical Supplier	earthing material de
Earth Electricals	098114 47799	Electrical Supplier	electrical earthing
NCR Enterprises	080 4606 5554	Electrical Supplier	electrical earthing
Capstan Super Enamelled Copper and	098116 43058	Electrical Supplier	copper earthing elec
Earthing World Marconite Earthing -	nan	Specialized Earthing	electrical earthing
Grover Electric & Trading Corporati	093199 08944	Electrical Supplier	electrical earthing
Superb Electronics	098109 59698	Electrical Supplier	electrical earthing
Reliable Earthing Solution	097180 00802	Specialized Earthing	earthing electrode s
Capstan Super Enamelled Copper and	098116 43058	Electrical Supplier	copper earthing elec
SHREE GEETANAND ENTERPRISES	098104 77872	Electrical Supplier	earthing material de
SBM Electric Techno Pvt. Ltd.	099716 16457	Electrical Supplier	earthing material de
New Gurgaon Traders	099582 84002	Electrical Supplier	earthing material de
LEEWO Contracts Pvt. Ltd.	0129 417 0003	Electrical Supplier	electrical earthing

Table A.16: Extracted Lead Data (Contd.)

Business Entity	Contact Number	Category	Source Query
Super electrical works	070116 84393	Electrical Supplier	electrical earthing
VEEWELL INDUSTRIES PRIVATE LIMITED	1800 572 1780	Electrical Supplier	earthing electrode s
E Control Devices - Thermal Managem	098106 14803	Electrical Supplier	electrical earthing
SHAKTI WINDING WIRES	098115 35859	Electrical Supplier	copper earthing elec
SBS Power & Earthing Solutions | Ea	093501 47264	Specialized Earthing	earthing electrode s
Anugrah Electronics & Appliances	011 2704 4500	Electrical Supplier	electrical earthing
SINGH EARTHING SYSTEM	097189 43900	Specialized Earthing	GI earthing electrod
AAYUSH ENTERPRISES	097170 87012	Electrical Supplier	electrical earthing
Arora electricals and Hardware stor	099106 25658	Electrical Supplier	electrical earthing
Ozone India | Laxmi Hardware And We	098104 98405	Electrical Supplier	earthing material de
Earthing World Marconite Earthing -	nan	Specialized Earthing	electrical earthing
BestofElectricals.com, online elect	011 4724 3000	Electrical Supplier	electrical earthing
SHREE GEETANAND ENTERPRISES	098104 77872	Electrical Supplier	earthing material de
Superb Electronics	098109 59698	Electrical Supplier	electrical earthing

Table A.17: Extracted Lead Data (Contd.)

Business Entity	Contact Number	Category	Source Query
As Electricals - Coppers and Earthi	095409 10091	Specialized Earthing	earthing electrode s
Ra-Vi Electricals	098102 46443	Electrical Supplier	earthing material de
Ra-Vi Electricals	098102 46443	Electrical Supplier	earthing material de
E Control Devices - Thermal Managem	098106 14803	Electrical Supplier	electrical earthing
SINGH EARTHING SYSTEM	097189 43900	Specialized Earthing	GI earthing electrod
P.T POWER SOLUTIONS	098113 10076	Electrical Supplier	electrical earthing
Star Electricals	098111 29851	Electrical Supplier	electrical earthing
Ozone India | Laxmi Hardware And We	098104 98405	Electrical Supplier	earthing material de
Reliable Earthing Solution	097180 00802	Specialized Earthing	earthing electrode s
Mohan Lal Munna Lal	080 4860 3248	Electrical Supplier	copper bonded earthi
Haryana Iron Store	098684 11322	Electrical Supplier	earthing material de
Ra-Vi Electricals	098102 46443	Electrical Supplier	earthing material de
SBM Electric Techno Pvt. Ltd.	099716 16457	Electrical Supplier	earthing material de
Earthing Material Supplier HR ENGIN	084485 61247	Specialized Earthing	earthing material de

Table A.18: Extracted Lead Data (Contd.)

Business Entity	Contact Number	Category	Source Query
Star Electricals	098111 29851	Electrical Supplier	electrical earthing
Ashoka Earthing Systems	nan	Specialized Earthing	earthing electrode s
SUBTLE CONTROLS I PVT LTD	0129 402 9540	Electrical Supplier	electrical earthing
ANSOL PREMIER INDUSTRIES LLP, Solde	099999 99752	Electrical Supplier	copper bonded earthi
Bhagwati Sales Corporation | electr	093152 61818	Electrical Supplier	earthing material de
Safreena Earthing Solution	098102 89947	Specialized Earthing	earthing electrode s
Bhagwati Sales Corporation | electr	093152 61818	Electrical Supplier	earthing material de
ROKSNA INDIA Pvt. Ltd.	097170 70100	Electrical Supplier	earthing material de
As Electricals - Coppers and Earthi	095409 10091	Specialized Earthing	earthing electrode s
SINGH EARTHING SYSTEM	097189 43900	Specialized Earthing	GI earthing electrod
Reliable Earthing Solution	097180 00802	Specialized Earthing	earthing electrode s
Electrical Traders	098110 67948	Electrical Supplier	electrical earthing
P.T POWER SOLUTIONS	098113 10076	Electrical Supplier	electrical earthing
Elvee Electricals	098680 78979	Electrical Supplier	electrical earthing

Table A.19: Extracted Lead Data (Contd.)

Business Entity	Contact Number	Category	Source Query
Sharma Electricals - Best Electrica	099583 63666	Electrical Supplier	electrical earthing
Standard Electric Co.	0129 400 6818	Electrical Supplier	electrical earthing
NV Electrical – Electrician | AC Re	075033 36203	Electrical Supplier	electrical earthing
Maruti Electrode	085954 82517	Electrical Supplier	earthing material de
Star Earthing International	099995 57452	Specialized Earthing	electrical earthing
SUBTLE CONTROLS I PVT LTD	0129 402 9540	Electrical Supplier	electrical earthing
Ra-Vi Electricals	098102 46443	Electrical Supplier	earthing material de
S K Super Enamelled Copper Wire	011 5621 0250	Electrical Supplier	copper earthing elec
VARDAAN TRADERS	nan	Electrical Supplier	earthing material de
GS ELECTROCONTROLS PVT LTD	099998 86013	Electrical Supplier	GI earthing electrod
Voltmen Energy Solutions Private Li	095400 03075	Electrical Supplier	earthing material de
Madan Lal Nanda & Sons	098737 86175	Electrical Supplier	copper bonded earthi
Karytron Electricals Private Limite	093155 20262	Electrical Supplier	electrical earthing
ROKSNA INDIA Pvt. Ltd.	097170 70100	Electrical Supplier	earthing material de

Table A.20: Extracted Lead Data (Contd.)

Business Entity	Contact Number	Category	Source Query
Ennob Infraa Solution	088822 22777	Electrical Supplier	GI earthing electrod
Reliable Earthing Solution	097180 00802	Specialized Earthing	earthing electrode s
Ra-Vi Electricals	098102 46443	Electrical Supplier	earthing material de
SBS Power & Earthing Solutions | Ea	093501 47264	Specialized Earthing	earthing electrode s
ROKSNA INDIA Pvt. Ltd.	097170 70100	Electrical Supplier	earthing material de
AKAALTECH,MAINTENANCE FREE EARTHING	099996 31369	Specialized Earthing	earthing electrode s
S.K. Refrigeration	098118 83216	Electrical Supplier	electrical earthing
Dorset, Hettich & Hafele - B S Hard	099109 91577	Electrical Supplier	earthing material de
SHIV SHAKTI BUILDING MATERIAL SUPPL	nan	Electrical Supplier	earthing material de
Shiva Sales Corporation Hardware St	098999 06521	Electrical Supplier	earthing material de
Anugrah Electronics & Appliances	011 2704 4500	Electrical Supplier	electrical earthing
Earthing Material Supplier HR ENGIN	084485 61247	Specialized Earthing	earthing material de
ANSOL PREMIER INDUSTRIES LLP, Solde	099999 99752	Electrical Supplier	copper bonded earthi
Standard Electric Co.	0129 400 6818	Electrical Supplier	electrical earthing

Table A.21: Extracted Lead Data (Contd.)

Business Entity	Contact Number	Category	Source Query
Anugrah Electronics & Appliances	011 2704 4500	Electrical Supplier	electrical earthing
Capstan Super Enamelled Copper and	098116 43058	Electrical Supplier	copper earthing elec
NCR Enterprises	080 4606 5554	Electrical Supplier	electrical earthing
NCR Enterprises	080 4606 5554	Electrical Supplier	electrical earthing
JAI HIND KISHAN AGRO	093100 05055	Electrical Supplier	earthing material de
VEEWELL INDUSTRIES PRIVATE LIMITED	1800 572 1780	Electrical Supplier	earthing electrode s
Ess Ess International	011 2362 8092	Electrical Supplier	electrical earthing
Bharat Electrical Industries	080 4860 2935	Electrical Supplier	electrical earthing
SPKN INDIA	099995 01843	Electrical Supplier	electrical earthing
Electrical Traders	098110 67948	Electrical Supplier	electrical earthing
Swasthi Earthing Solutions	080 4801 8868	Specialized Earthing	copper bonded earthi
Ennob Infraa Solution	088822 22777	Electrical Supplier	GI earthing electrod
Capstan Super Enamelled Copper and	098116 43058	Electrical Supplier	copper earthing elec
AKAALTECH,MAINTENANCE FREE EARTHING	099996 31369	Specialized Earthing	earthing electrode s

Table A.22: Extracted Lead Data (Contd.)

Business Entity	Contact Number	Category	Source Query
ROKSNA INDIA Pvt. Ltd.	097170 70100	Electrical Supplier	earthing material de
Bhagwati Sales Corporation | electr	093152 61818	Electrical Supplier	earthing material de
ROKSNA INDIA Pvt. Ltd.	097170 70100	Electrical Supplier	earthing material de
super electronics pvt ltd	085100 06341	Electrical Supplier	electrical earthing
Maruti Electrode	085954 82517	Electrical Supplier	earthing material de
BestofElectricals.com, online elect	011 4724 3000	Electrical Supplier	electrical earthing
JAI HIND KISHAN AGRO	093100 05055	Electrical Supplier	earthing material de
S.K. Refrigeration	098118 83216	Electrical Supplier	electrical earthing
Kartar Singh & Co.	098990 45610	Electrical Supplier	earthing electrode s
Mohan Lal Munna Lal	080 4860 3248	Electrical Supplier	copper bonded earthi
SHREE GEETANAND ENTERPRISES	098104 77872	Electrical Supplier	earthing material de
Ashoka Earthing Systems	nan	Specialized Earthing	earthing electrode s
Superb Electronics	098109 59698	Electrical Supplier	electrical earthing
super electronics pvt ltd	085100 06341	Electrical Supplier	electrical earthing

Table A.23: Extracted Lead Data (Contd.)

Business Entity	Contact Number	Category	Source Query
Voltmen Energy Solutions Private Li	095400 03075	Electrical Supplier	earthing material de
Dorset, Hettich & Hafele - B S Hard	099109 91577	Electrical Supplier	earthing material de
SBS Power & Earthing Solutions | Ea	093501 47264	Specialized Earthing	earthing electrode s
SPKN INDIA	099995 01843	Electrical Supplier	electrical earthing
Capstan Super Enamelled Copper and	098116 43058	Electrical Supplier	copper earthing elec
Standard Electric Co.	0129 400 6818	Electrical Supplier	electrical earthing
Kartar Singh & Co.	098990 45610	Electrical Supplier	earthing electrode s
SHREE GEETANAND ENTERPRISES	098104 77872	Electrical Supplier	earthing material de
Shiva Sales Corporation Hardware St	098999 06521	Electrical Supplier	earthing material de
Haryana Iron Store	098684 11322	Electrical Supplier	earthing material de
SHIV SHAKTI BUILDING MATERIAL SUPPL	nan	Electrical Supplier	earthing material de
SHREE GEETANAND ENTERPRISES	098104 77872	Electrical Supplier	earthing material de
Earthing Material Supplier HR ENGIN	084485 61247	Specialized Earthing	earthing material de
AKAALTECH,MAINTENANCE FREE EARTHING	099996 31369	Specialized Earthing	earthing electrode s

Table A.24: Extracted Lead Data (Contd.)

Business Entity	Contact Number	Category	Source Query
SHAKTI WINDING WIRES	098115 35859	Electrical Supplier	copper earthing elec
Bhagwati Sales Corporation | electr	093152 61818	Electrical Supplier	earthing material de
Gleam Light India Pvt. Ltd.	088003 86366	Electrical Supplier	electrical earthing
Standard Electric Co.	0129 400 6818	Electrical Supplier	electrical earthing
Arora electricals and Hardware stor	099106 25658	Electrical Supplier	electrical earthing
SBS Power & Earthing Solutions | Ea	093501 47264	Specialized Earthing	earthing electrode s
Super electrical works	070116 84393	Electrical Supplier	electrical earthing
Earthing Material Supplier HR ENGIN	084485 61247	Specialized Earthing	earthing material de
BHANU URJA SOLAR GREEN ENERGY- Expe	092116 85257	Electrical Supplier	earthing material de
Power Factor Shop	097170 05541	Electrical Supplier	electrical earthing
Karytron Electricals Private Limite	093155 20262	Electrical Supplier	electrical earthing
AKAALTECH,MAINTENANCE FREE EARTHING	099996 31369	Specialized Earthing	earthing electrode s
K.K Tools & Electric Corporation	080 4605 3958	Electrical Supplier	copper bonded earthi
New Samrat Electronics	097101 51313	Electrical Supplier	electrical earthing

Table A.25: Extracted Lead Data (Contd.)

APPENDIX B: CORE SYSTEM SOURCE CODE

This appendix contains the core programmatic logic driving both the LeadScribe scraper and the frontend dashboard. The code demonstrates the use of Playwright for automation and Next.js for UI rendering.

B.1 LeadScribe Python Engine

B.1 LeadScribe Python Engine

```
import pandas as pd
import time
import os
from playwright.sync_api import sync_playwright

OUTPUT_CSV = "earthing_electrode_resellers.csv"

SEARCH_QUERIES = [
    "earthing electrode dealers Delhi",
    "earthing electrode suppliers Delhi NCR",
    "electrical earthing shops Delhi",
    "copper earthing electrode resellers Delhi",
    "GI earthing electrode dealers Noida",
    "earthing material dealers Gurugram",
    "electrical earthing dealers Faridabad",
    "copper bonded earthing rod dealers Delhi",
    "earthing electrode distributors Ghaziabad",
    "electrical earthing suppliers Chandigarh",
    "earthing electrode shops Ludhiana",
    "earthing material suppliers Jaipur",
    "copper earthing electrode dealers Amritsar",
    "electrical earthing dealers Agra",
    "earthing electrode dealers Meerut",
    "earthing electrode suppliers Dehradun",
    "earthing electrode dealers Lucknow",
    "electrical earthing material dealers North India",
    "maintenance free earthing electrode dealers Delhi",
    "chemical earthing electrode resellers Delhi",
]

def is_manufacturer(name):
    keywords = ['manufacturer', 'manufacturing', 'mfg', 'fabricat', 'factory', 'production
unit']
    return any(kw in name.lower() for kw in keywords)

def record_exists(name, phone, seen_set):
    key = (name.strip().lower(), phone.strip())
    if key in seen_set:
        return True
    seen_set.add(key)
    return False

def append_to_csv(record):
    df = pd.DataFrame([record])
    file_exists = os.path.exists(OUTPUT_CSV)
    df.to_csv(OUTPUT_CSV, mode='a', header=not file_exists, index=False)

def scrape_query(query, browser, seen_set, target_per_query=15):
    results = []
    search_url = f"https://www.google.com/maps/search/{query.replace(' ', '+')}"
    search_page = browser.new_page()

    try:
        print(f"\n[SEARCH] {query}")
        search_page.goto(search_url, timeout=60000)
        try:
            search_page.wait_for_selector('a[href*="https://www.google.com/maps/place/"]',
timeout=25000)

            time.sleep(3)
        except:
            print("  No results found, skipping.")
            search_page.close()
            return results
```

```
        processed_urls = set()
        scroll_attempts = 0
        empty_scrolls = 0

        while len(results) < target_per_query and scroll_attempts < 60:
            listings =
search_page.locator('a[href*="https://www.google.com/maps/place/"]').all()
            new_urls = []
            for loc in listings:
                url = loc.get_attribute('href')
                if url and url not in processed_urls:
                    new_urls.append(url)
                    processed_urls.add(url)

            if not new_urls:
                empty_scrolls += 1
                if empty_scrolls >= 5:
                    print("  No new listings after scrolling. Moving on.")
                    break
            else:
                empty_scrolls = 0

            for url in new_urls:
                if len(results) >= target_per_query:
                    break

                detail_page = browser.new_page()
                try:
                    detail_page.goto(url, timeout=30000)
                    detail_page.wait_for_load_state('domcontentloaded')
                    time.sleep(2)

                    name_el = detail_page.locator('h1').first
                    name = name_el.inner_text().strip() if name_el.count() > 0 else ""
                    if not name:
                        detail_page.close()
                        continue

                    if is_manufacturer(name):
                        print(f"  [SKIP - Manufacturer] {name}")
                        detail_page.close()
                        continue

                    phone_el = detail_page.locator('button[data-item-id^="phone:tel:"]')
                    phone = ""
                    if phone_el.count() > 0:
                        phone = phone_el.first.inner_text().replace('\ue0b0',
'').replace('\n', '').strip()

                    address_el = detail_page.locator('button[data-item-id="address"]')


                    address = address_el.first.inner_text().strip() if address_el.count() > 0
else ""

                    if record_exists(name, phone, seen_set):
                        print(f"  [DUPLICATE] {name}")
                        detail_page.close()
                        continue

                    record = {
                        'Company Name': name,
                        'Phone Number': phone,
                        'Address': address,
                    }
                    results.append(record)
                    append_to_csv(record)
                    safe_name = name.encode('ascii', errors='replace').decode()
                    safe_phone = phone.encode('ascii', errors='replace').decode()
                    safe_addr = address[:50].encode('ascii', errors='replace').decode()
                    print(f"  [SAVED] {safe_name} | {safe_phone} | {safe_addr}")

                except Exception as ex:
                    print(f"  [ERROR] {ex}")
                finally:
                    detail_page.close()
```

```
            # Scroll to load more
            try:
                feed = search_page.locator('div[role="feed"]')
                if feed.count() > 0:
                    feed.last.evaluate("el => el.scrollBy(0, 2000)")
                else:
                    search_page.keyboard.press('PageDown')
                    search_page.keyboard.press('PageDown')
                time.sleep(2)
            except:
                pass
            scroll_attempts += 1

    except Exception as ex:
        print(f"[ERROR] Query '{query}': {ex}")
    finally:
        search_page.close()

    return results


def main():
    total_saved = 0
    seen_set = set()

    if os.path.exists(OUTPUT_CSV):
        existing = pd.read_csv(OUTPUT_CSV)
        for _, row in existing.iterrows():
            key = (str(row.get('Company Name', '')).strip().lower(), str(row.get('Phone
Number', '')).strip())
            seen_set.add(key)
        total_saved = len(existing)


        print(f"[RESUME] Found {total_saved} existing records. Continuing.")

    with sync_playwright() as p:
        browser = p.chromium.launch_persistent_context(
            user_data_dir=r"C:\Users\aditya
tyagi\OneDrive\Desktop\antigravity\earthing_scraper\chrome_profile2",
            headless=False,
            args=["--disable-blink-features=AutomationControlled"],
            ignore_default_args=["--enable-automation"],
            user_agent="Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,
like Gecko) Chrome/120.0.0.0 Safari/537.36"
        )

        login_page = browser.pages[0] if browser.pages else browser.new_page()
        login_page.goto("https://accounts.google.com/", timeout=60000)
        print("\n>>> ACTION REQUIRED: Log in to Google in the browser window.")
        print(">>> Waiting 45 seconds... (wait if already logged in)\n")
        login_page.wait_for_timeout(45000)
        login_page.goto("about:blank")

        for query in SEARCH_QUERIES:
            if total_saved >= 200:
                print(f"\nTarget of 200+ reached! Stopping.")
                break
            per_query_target = min(20, max(10, 200 - total_saved + 10))
            data = scrape_query(query, browser, seen_set, target_per_query=per_query_target)
            total_saved += len(data)
            print(f"  --> Total so far: {total_saved}")

        browser.close()

    print(f"\n{'='*50}")
    print(f"DONE! {total_saved} contacts saved to {OUTPUT_CSV}")
    print(f"{'='*50}")


if __name__ == "__main__":
    main()
```

B.2 Next.js Dashboard UI Components

```
'use client';

import { useState, useEffect } from 'react';
import {
  Search,
  Terminal,
  Database,
  Zap,
  Download,
  Plus,
  CheckCircle,
  AlertCircle,
  Clock,
  ExternalLink,
  Phone,
  Mail,
  MapPin
} from 'lucide-react';

export default function ScraperDashboard() {
  const [leads, setLeads] = useState<any[]>([]);
  const [loading, setLoading] = useState(true);
  const [status, setStatus] = useState('Idle');
  const [queries, setQueries] = useState([
    "earthing electrode dealers Delhi",
    "earthing electrode suppliers Delhi NCR",
    "electrical earthing shops Delhi"
  ]);
  const [newQuery, setNewQuery] = useState('');
  const [logs, setLogs] = useState([
    "[SYSTEM] Scraper Engine Initialized.",
    "[SYSTEM] Ready for new queries."
  ]);

  useEffect(() => {
    fetchLeads();
  }, []);

  const fetchLeads = async () => {
    try {
      const res = await fetch('/api/leads');
      const data = await res.json();
      if (Array.isArray(data)) {
        setLeads(data);
      }
    } catch (err) {
      console.error(err);
    } finally {
      setLoading(false);
    }
  };

  const addQuery = () => {
    if (newQuery.trim()) {
      setQueries([...queries, newQuery.trim()]);
      setNewQuery('');
      setLogs(prev => [...prev, `[QUERY] Added: ${newQuery.trim()}`]);
    }
  };

  const startScraper = () => {
    setStatus('Running');
    setLogs(prev => [...prev, `[ACTION] Starting scraper for ${queries.length} queries...`]);

    // Simulating progress
    setTimeout(() => {
      setLogs(prev => [...prev, `[SCRAPER] Initializing browser context...`]);
      setTimeout(() => {
        setLogs(prev => [...prev, `[SCRAPER] Searching: ${queries[0]}`]);
        setTimeout(() => {
          setLogs(prev => [...prev, `[SCRAPER] Found 15 leads. Bypassing manufacturers...`]);
        }, 2000);
      }, 2000);
```

```
    }, 1000);
  };

  return (
    <div className="min-h-screen bg-[#0a0a0a] text-[#f8f8f8] p-8 font-sans selection:bg-
[#00f2ff] selection:text-black">


      {/* Header */}
      <header className="flex justify-between items-center mb-12">
        <div>
          <h1 className="text-4xl font-black tracking-tighter flex items-center gap-3">
            <Zap className="text-[#00f2ff] fill-[#00f2ff] animate-pulse" size={32} />
            LEAD<span className="text-[#bc13fe]">SCRIBE</span> AI
          </h1>
          <p className="text-gray-500 font-medium">Professional Lead Generation Dashboard for
Gmax Electric</p>
        </div>
        <div className="flex gap-4">
          <button
            onClick={startScraper}
            disabled={status === 'Running'}
            className="neon-button px-6 py-3 font-bold rounded-xl flex items-center gap-2
disabled:opacity-50"
          >
            {status === 'Running' ? <Clock className="animate-spin" /> : <Zap size={18} />}
            {status === 'Running' ? 'Scraping...' : 'Start Engine'}
          </button>
          <button className="glass px-6 py-3 font-bold rounded-xl border border-white/10
hover:border-white/20 transition-all flex items-center gap-2">
            <Download size={18} />
            Export CSV
          </button>
        </div>
      </header>

      <main className="grid grid-cols-12 gap-8">
        {/* Left Column: Stats & Queries */}
        <div className="col-span-12 lg:col-span-4 space-y-8">
          {/* Stats Grid */}
          <div className="grid grid-cols-2 gap-4">
            <div className="glass p-6 border-l-4 border-l-[#00f2ff]">
              <p className="text-gray-500 text-xs font-bold uppercase tracking-widest mb-
1">Total Leads</p>
              <h3 className="text-3xl font-black">{leads.length}</h3>
            </div>
            <div className="glass p-6 border-l-4 border-l-[#bc13fe]">
              <p className="text-gray-500 text-xs font-bold uppercase tracking-widest mb-
1">Queries</p>
              <h3 className="text-3xl font-black">{queries.length}</h3>
            </div>
          </div>

          {/* Query Management */}


          <div className="glass p-6 space-y-4">
            <h4 className="font-bold flex items-center gap-2">
              <Search size={18} className="text-[#00f2ff]" />
              Target Queries
            </h4>
            <div className="space-y-2 max-h-[300px] overflow-y-auto pr-2">
              {queries.map((q, i) => (
                <div key={i} className="bg-white/5 p-3 rounded-lg text-sm border border-
white/5 flex justify-between items-center group">
                  <span className="text-gray-300">{q}</span>
                  <CheckCircle size={14} className="text-[#00f2ff] opacity-0 group-
hover:opacity-100 transition-opacity" />
                </div>
              ))}
            </div>
            <div className="flex gap-2">
              <input
                type="text"
                value={newQuery}
                onChange={(e) => setNewQuery(e.target.value)}
                placeholder="Add new query..."
```

```
                className="bg-white/5 border border-white/10 rounded-lg px-4 py-2 text-sm
flex-1 focus:outline-none focus:border-[#00f2ff] transition-colors"
              />
              <button
                onClick={addQuery}
                className="bg-white/10 p-2 rounded-lg hover:bg-white/20 transition-colors"
              >
                <Plus size={20} />
              </button>
            </div>
          </div>

          {/* Log Viewer */}
          <div className="glass p-6 space-y-4 bg-black/40">
            <h4 className="font-bold flex items-center gap-2">
              <Terminal size={18} className="text-[#bc13fe]" />
              Terminal Logs
            </h4>
            <div className="bg-black p-4 rounded-lg font-mono text-xs space-y-2 h-[200px]
overflow-y-auto border border-white/5">
              {logs.map((log, i) => (
                <div key={i} className={log.startsWith('[ERROR]') ? 'text-red-400' :
log.startsWith('[SYSTEM]') ? 'text-blue-400' : 'text-green-400'}>
                  <span className="text-gray-600 mr-2">[{new
Date().toLocaleTimeString()}]</span>

                  {log}
                </div>
              ))}
            </div>
          </div>
        </div>

        {/* Right Column: Results Table */}
        <div className="col-span-12 lg:col-span-8">
          <div className="glass p-6 h-full border border-white/5">
            <div className="flex justify-between items-center mb-6">
              <h4 className="font-bold flex items-center gap-2 text-xl">
                <Database size={20} className="text-[#00f2ff]" />
                Extracted Leads
              </h4>
              <div className="text-xs text-gray-500 font-mono">Showing {leads.length}
records</div>
            </div>

            <div className="overflow-x-auto">
              <table className="w-full text-left">
                <thead>
                  <tr className="text-gray-500 text-xs font-bold uppercase tracking-widest
border-b border-white/10">
                    <th className="pb-4 px-4">Company</th>
                    <th className="pb-4 px-4">Contact</th>
                    <th className="pb-4 px-4">Address</th>
                    <th className="pb-4 px-4">Query</th>
                  </tr>
                </thead>
                <tbody className="text-sm">
                  {loading ? (
                    <tr>
                      <td colSpan={4} className="py-12 text-center text-gray-500">
                        <Zap className="animate-spin inline-block mr-2" />
                        Loading leads from engine...
                      </td>
                    </tr>
                  ) : leads.map((lead, i) => (
                    <tr key={i} className="border-b border-white/5 hover:bg-white/[0.02]
transition-colors group">
                      <td className="py-4 px-4">
                        <div className="font-bold group-hover:text-[#00f2ff] transition-
colors">{lead['Company Name']}</div>
                        <div className="text-[10px] text-gray-600 uppercase font-black
tracking-tighter">{lead['Category']}</div>

                      </td>
                      <td className="py-4 px-4 space-y-1">
```

```
                        {lead['Phone Number'] && (
                          <div className="flex items-center gap-2 text-gray-400 group-
hover:text-white transition-colors">
                            <Phone size={12} className="text-[#00f2ff]" />
                            {lead['Phone Number']}
                          </div>
                        )}
                        {lead['Email ID'] && (
                          <div className="flex items-center gap-2 text-gray-400 group-
hover:text-white transition-colors">
                            <Mail size={12} className="text-[#bc13fe]" />
                            {lead['Email ID'].split(',')[0]}
                          </div>
                        )}
                      </td>
                      <td className="py-4 px-4">
                        <div className="flex items-start gap-2 text-gray-400 max-w-[200px]
truncate">
                          <MapPin size={12} className="mt-1 text-[#ff007f]" />
                          <span className="text-xs">{lead['Address']}</span>
                        </div>
                      </td>
                      <td className="py-4 px-4">
                        <span className="bg-white/5 px-2 py-1 rounded text-[10px] text-gray-
500 group-hover:bg-[#00f2ff]/10 group-hover:text-[#00f2ff] transition-all">
                          {lead['Search Query']}
                        </span>
                      </td>
                    </tr>
                  ))}
                </tbody>
              </table>
            </div>
          </div>
        </div>
      </main>

      {/* Footer */}
      <footer className="mt-12 text-center text-gray-600 text-xs border-t border-white/5 pt-
8">
        &copy; 2026 Gmax Electric AI Labs | Built by Aditya Tyagi | USICT Internship Final
Submission
      </footer>
    </div>
  );
}
```